



Faculty of Engineering and Technology
Electrical & Computer Engineering Department

ENCS4370

Computer Architecture

First Semester (2025/2026)

Project #2

Design and Verification of a Simple Multi Cycle RISC
Processor in Verilog

Prepared by:

Israa Alyazouri	#1231612	Section 3
Deema Jaar	#1242479	Section 1
Yasmin Badwan	#1230271	Section 1

Instructor: Dr. Aziz Qaroush

Date: June 22, 2026

Abstract

This report documents the complete design and verification of a 32-bit multicycle RISC processor implemented in Verilog HDL and simulated with Aldec Active-HDL. The processor supports 20 instructions across R-type, I-type, and J-type formats, operating on 16 general-purpose 32-bit registers with R0 hardwired to zero, and a Harvard memory model with separate 256×32-bit instruction and data memories. A Moore Finite State Machine (FSM) sequences up to five pipeline stages per instruction (IF, ID, EX, MEM, WB), achieving a CPI range of 2 to 5. Six inter-stage registers — IR, A, B, ALUOut, MDR, and PCsave — isolate each stage from combinational instability in adjacent stages. All modules are verified individually through dedicated testbenches before full-processor integration testing.

Table of Contents

Abstract.....	II
Table of Figures.....	V
List of Tables.....	VI
Introduction.....	1
1. Design and Implementation	2
1.1 Processor Specifications	2
1.2 Instruction Set Architecture.....	3
1.2.1 Instruction Formats.....	3
1.2.2 Instruction Encoding Table.....	4
1.2.3 RTL Micro-Operations.....	6
1.3 Datapath Design.....	7
1.3.1 Program Counter (PC) and the PCUpdate / PCSrc Logic.....	8
1.3.2 Instruction Memory and the Reason IR Exists.....	10
1.3.3 Instruction Decoder	10
1.3.4 Register File 16 Registers, Two Read Ports, One Write Port	11
1.3.5 Buffers A and B Protecting Read Values from Stage Interference.....	11
1.3.6 Sign/Zero Extender Making 18-bit and 26-bit Constants ALU-Ready.....	12
1.3.7 ALU-B Multiplexer (ALUSrc) and the ALU	14
1.3.8 ALUOut The Indispensable Address Safety Buffer	15
1.3.9 Data Memory The Harvard-Separated R/W Store.....	16
1.3.10 MDR (Memory Data Register) Protecting the Loaded Value into WB	16
1.3.11 The Memories Compared Side by Side.....	17
1.3.12 The Four Multiplexers Where Control Signals Meet Data Paths	18
1.4 Control Unit FSM Design.....	20
1.4.1 FSM Architecture.....	20
1.4.2 Next-State Transition Table.....	21
1.4.3 Output Logic Complete Control Signal Table.....	22
1.5 Boolean Equations of Key Control Signals.....	22
1.5.1 PC Update Enable.....	22
1.5.2 PCSrc Multiplexer Select	23

1.5.3 Branch-Taken Logic	23
1.5.4 State-Decoded Single-Bit Signal Equations	24
1.5.5 State-Decoded Single-Bit Signal Equations	24
1.5.6 ALUSrc and ALUOp Equations	25
2. Testing and Simulation	26
2.1 R-Type Instructions Test	27
2.1.1 Final Result:	38
2.2 I-Type Instructions Test.....	39
2.2.1 Final Result:	48
2.3 J-Type Instructions Test	49
2.3.1 Final Result:	52
3. Waveform Analysis	53
3.1 R-Type Instructions Waveform	53
3.2 I-Type Instructions Waveform.....	54
3.3 J-Type Instructions Waveform	55
Challenges.....	56
Teamwork	57
Conclusion	58

Table of Figures

Figure 1: Complete Datapath.....	7
Figure 2 :PC Update Enable Logic	22
Figure 3 :Branch Decision and PCSrc Control Logic.....	23
Figure 4 : Complete FSM Output Logic.....	24
Figure 5: ALUSrc and ALUOp Control Equations.....	25
Figure 6 :ADD R3, R1, R2	28
Figure 7 : SUB R4, R1, R2	29
Figure 8 : AND R5, R1, R2	30
Figure 9 : OR R6, R1, R2	31
Figure 10 : XOR R7, R1, R2.....	32
Figure 11 : NOR R8, R1, R2.....	33
Figure 12 : SLL R9, R2, R2	34
Figure 13 : SRL R10, R1, R2.....	35
Figure 14 : CLR R11.....	36
Figure 15 : JR R12	37
Figure 16: ADDI R3, R1, 5	40
Figure 17: ANDI R4, R1, 3	41
Figure 18: ORI R5, R1, 3.....	42
Figure 19: LW R6, 2(R13).....	43
Figure 20: SW R2, 3(R13).....	44
Figure 21: SWAP R7, 4(R13)	45
Figure 22: BEQ R1, R1, 1.....	46
Figure 23: BNE R9, R1, 1.....	47
Figure 24: J 34	50
Figure 25: JAL 36	51
Figure 26: R-Type Instructions Waveform	53
Figure 27 : I-Type, Memory, and Branch Instructions Waveform.....	54
Figure 28 : J-Type Instructions Waveform.....	55
Figure 29 : Teamwork Contribution Distribution	57

List of Tables

Table 1:Processor Architectural Parameters	2
Table 2:R-Type Instruction Format.....	3
Table 3:I-Type Instruction Format	3
Table 4: J-Type Format	4
Table 5:Complete Instruction Encoding	4
Table 6: RTL Micro-Operations.....	6
Table 7:PC MUX Selection Logic	9
Table 8:Extender Mode Selection.	13
Table 9:ALU Operations.....	14
Table 10:Memory Comparison.	17
Table 11:Multiplexer Summary.....	18
Table 12 : FSM Next-State Transition Table	21
Table 13: Complete FSM Output Logic.....	22
Table 14 : PCSrc encoding and source selection.	23
Table 15:WBData encoding.	24
Table 16 : RegDst encoding.	25
Table 17 :R Type Instructions program.....	27
Table 18 : I Type Instructions Test program	39
Table 19:J Type Instructions Test program	49

Introduction

Computer architecture defines how hardware components are organized and how instructions are executed at the processor level. This project focuses on the design and verification of a 32-bit multicycle RISC processor implemented using Verilog HDL, following a custom instruction set architecture (ISA). The processor supports three instruction formats: R-type, I-type, and J-type, which collectively cover arithmetic, logical, memory access, branch, jump, and link operations. Each instruction is executed through a sequence of well-defined stages, including instruction fetch, decode, execute, memory access, and write-back, allowing efficient reuse of hardware resources across different instruction types. The design is based on a structured datapath that integrates key components such as the program counter, instruction register, register file, arithmetic logic unit (ALU), sign/zero extender, data and instruction memories, multiplexers, and intermediate pipeline registers. A Moore finite state machine (FSM) is used as the control unit to generate precise control signals that coordinate the execution of each instruction at every clock cycle. The processor was implemented and verified using simulation to ensure correct functionality of all instructions and proper interaction between the datapath and control unit. The project demonstrates the relationship between ISA design, datapath organization, control logic design, and hardware-level verification in a complete processor system.

1. Design and Implementation

1.1 Processor Specifications

The processor is a 32-bit word-addressed multicycle RISC machine with fixed instruction width. All architectural parameters are fixed at elaboration time. The multicycle approach assigns each instruction its own variable-length sequence of stages, allowing fast instructions (J: 2 cycles) to complete quickly while complex ones (LW: 5 cycles) use only the hardware they need. Resource sharing between stages. The ALU doubles as an address calculator and a branch comparator eliminates redundant functional units without increasing CPI for unaffected instructions.

Table 1: Processor Architectural Parameters

Parameter	Specification	Design Rationale
Data / Instruction width	32 bits	Uniform width simplifies fetch logic; one memory read per instruction
General-purpose registers	16 (R0 – R15), 32-bit each	4-bit address fields; R0 hardwired to zero (write silently discarded)
Program Counter	32 bits, word-addressed	PC+1 = next sequential instruction; no byte-offset arithmetic needed
Memory model	Harvard (separate I-mem and D-mem)	Instruction fetch and data access can occur simultaneously without structural hazards
Memory depth	256 × 32-bit words each	PC[7:0] / ALUOut[7:0] used as address; upper bits ignored
Control architecture	Moore FSM, 5 states	Outputs depend only on state — no input-driven glitches reach the datapath mid-cycle
CPI range	2 – 5 cycles/instruction	J=2, CLR/JAL/BEQ/BNE/JR=3, R-type/I-ALU/SW=4, LW/SWAP=5
ISA size	20 instructions	6-bit opcode field (opcodes 0–19 used; 20–63 reserved for future extension)

1.2 Instruction Set Architecture

1.2.1 Instruction Formats

All 32-bit instructions share a fixed 6-bit opcode in bits [31:26], enabling a single-path combinational decode. The remaining 26 bits are partitioned per format. A key subtlety in I-type encoding is that IR[17:14] simultaneously serves as rt (source register for SW) and as the high 4 bits of the 18-bit immediate; the FSM resolves this overlap through ALUSrc and BWrite rather than duplicating hardware.

R-Type Register-Register Operations (Opcodes 0–9)

Table 2: R-Type Instruction Format

31–26	25–22	21–18	17–14	13–0
opcode [5:0]	rd [3:0] <i>destination</i>	rs [3:0] <i>source 1</i>	rt [3:0] <i>source 2</i>	unused [13:0] <i>zero-filled</i>

The 14 unused bits are not routed to any functional unit and are ignored by the datapath and control hardware.

I-Type Immediate / Memory / Branch (Opcodes 10–17)

Table 3: I-Type Instruction Format

31–26	25–22	21–18	17–14	13–0
opcode [5:0]	rd [3:0]	rs [3:0]	rt [3:0] <i>(also imm[17:14])</i>	immediate [17:0] <i>18-bit constant</i>

For SW, the rt field is used as a source register address, while for other I-Type instructions the 18-bit immediate is passed to the extender for sign or zero extension.

J-Type Unconditional Jump (Opcodes 18–19)

Table 4: J-Type Format

31–26	25–0
opcode [5:0]	offset [25:0] :26-bit signed absolute target address

The 26-bit offset is selected by ExtSel=1, sign-extended to 32 bits, and added to the current PC to generate JTA.

1.2.2 Instruction Encoding Table

Table 5: Complete Instruction Encoding

#	Mnemonic	Fmt	Opcode	RTL Operation	Stages	CPI
1	ADD	R	0	$R[rd] \leftarrow R[rs] + R[rt]$	IF·ID·EX·WB	4
2	SUB	R	1	$R[rd] \leftarrow R[rs] - R[rt]$	IF·ID·EX·WB	4
3	AND	R	2	$R[rd] \leftarrow R[rs] \& R[rt]$	IF·ID·EX·WB	4
4	OR	R	3	$R[rd] \leftarrow R[rs] R[rt]$	IF·ID·EX·WB	4
5	XOR	R	4	$R[rd] \leftarrow R[rs] \wedge R[rt]$	IF·ID·EX·WB	4
6	NOR	R	5	$R[rd] \leftarrow \sim(R[rs] R[rt])$	IF·ID·EX·WB	4
7	SLL	R	6	$R[rd] \leftarrow R[rs] \ll R[rt][4:0]$	IF·ID·EX·WB	4
8	SRL	R	7	$R[rd] \leftarrow R[rs] \gg R[rt][4:0]$	IF·ID·EX·WB	4
9	CLR	R	8	$R[rd] \leftarrow 0$ (no EX needed)	IF·ID·WB	3
10	JR	R	9	$PC \leftarrow R[rs]$	IF·ID·EX	3

#	Mnemonic	Fmt	Opcode	RTL Operation	Stages	CPI
11	ADDI	I	10	$R[rd] \leftarrow R[rs] + \text{SignExt}(\text{imm18})$	IF·ID·EX·WB	4
12	ANDI	I	11	$R[rd] \leftarrow R[rs] \& \text{ZeroExt}(\text{imm18})$	IF·ID·EX·WB	4
13	ORI	I	12	$R[rd] \leftarrow R[rs] \text{ZeroExt}(\text{imm18})$	IF·ID·EX·WB	4
14	LW	I	13	$R[rd] \leftarrow \text{Mem}[R[rs] + \text{SignExt}(\text{imm18})]$	IF·ID·EX·MEM·WB	5
15	SW	I	14	$\text{Mem}[R[rs] + \text{SignExt}(\text{imm18})] \leftarrow R[rt]$	IF·ID·EX·MEM	4
16	SWAP	I	15	$\text{tmp} \leftarrow R[rd];$ $R[rd] \leftarrow \text{Mem}[\text{addr}];$ $\text{Mem}[\text{addr}] \leftarrow \text{tmp}$	IF·ID·EX·MEM·WB	5
17	BEQ	I	16	if $R[rs] = R[rt]$: $\text{PC} \leftarrow (\text{PC}+1) + \text{SignExt}(\text{imm18})$	IF·ID·EX	3
18	BNE	I	17	if $R[rs] \neq R[rt]$: $\text{PC} \leftarrow (\text{PC}+1) + \text{SignExt}(\text{imm18})$	IF·ID·EX	3
19	J	J	18	$\text{PC} \leftarrow \text{SignExt}(\text{offset26})$	IF·ID	2
20	JAL	J	19	$R14 \leftarrow \text{PC}+1$; $\text{PC} \leftarrow \text{SignExt}(\text{offset26})$	IF·ID·WB	3

1.2.3 RTL Micro-Operations

Each cell represents one clock cycle. Concurrent register transfers within a single cycle are separated by semicolons.

Table 6: RTL Micro-Operations

Instruction(s)	Cycle 1 — IF	Cycle 2 — ID	Cycle 3 — EX	Cycle 4 — MEM	Cycle 5 — WB
ADD SUB AND OR XOR NOR SLL SRL	IR←Mem[PC]; PC←PC+1	A←R[rs]; B←R[rt]	ALUOut←A⊕B	—	R[rd]←ALUOut
ADDI ANDI ORI	IR←Mem[PC]; PC←PC+1	A←R[rs]; B←R[rt]	ALUOut←A⊕Imm32	—	R[rd]←ALUOut
LW	IR←Mem[PC]; PC←PC+1	A←R[rs]	ALUOut←A+Imm32	MDR←Mem[ALUOut]	R[rd]←MDR
SW	IR←Mem[PC]; PC←PC+1	A←R[rs]; B←R[rt]	ALUOut←A+Imm32	Mem[ALUOut]←B	—
SWAP	IR←Mem[PC]; PC←PC+1	A←R[rs]; B←R[rd]	ALUOut←A+Imm32	MDR←Mem[ALUOut]; Mem[ALUOut]←B	R[rd]←MDR
CLR	IR←Mem[PC]; PC←PC+1	—	—	—	R[rd]←0
JR	IR←Mem[PC]; PC←PC+1	A←R[rs]	PC←A	—	—
BEQ	IR←Mem[PC]; PC←PC+1	A←R[rs]; B←R[rt]	zero←(A−B=0); if zero: PC←PC+Imm32	—	—
BNE	IR←Mem[PC]; PC←PC+1	A←R[rs]; B←R[rt]	zero←(A−B=0); if ¬zero: PC←PC+Imm32	—	—
J	IR←Mem[PC]; PC←PC+1	PC←Imm32	—	—	—
JAL	IR←Mem[PC]; PC←PC+1	PCsave←PC; PC←Imm32	—	—	R14←PCsave

1.3 Datapath Design

The datapath is the complete physical infrastructure every storage element, functional unit, wire, and multiplexer through which data travels during instruction execution. It is implemented in `processor.v`, which instantiates every submodule and connects them together. The key architectural truth is that *the control unit carries no data*. It only raises and lowers one-bit control signals each clock cycle, and those signals tell each datapath component what to do. The datapath executes those instructions obediently. Designing it means answering, for every instruction group: what path does the data take, what does each component produce and consume, and why can no piece be removed without breaking something?

Data flows predominantly left to right. The Program Counter supplies the current instruction address to Instruction Memory. The memory output, a live combinational wire called `instruction_out` is latched by the Instruction Register (IR) so it remains stable throughout all remaining stages. The Instruction Decoder extracts named field wires from IR and routes them: opcode to the FSM, register addresses to the Register File, and the immediate field to the Sign/Zero Extender. Register values are buffered in A and B, flow through the ALU, whose result is frozen in `ALUOut`. For load/store instructions, `ALUOut` addresses Data Memory and the loaded value is buffered in MDR. Finally, the Write-Back MUX selects the correct value and drives it back into the Register File. Every buffer between stages exists for one concrete reason, a live wire that would change value before the next stage could safely use it. The figure below shows the complete datapath.

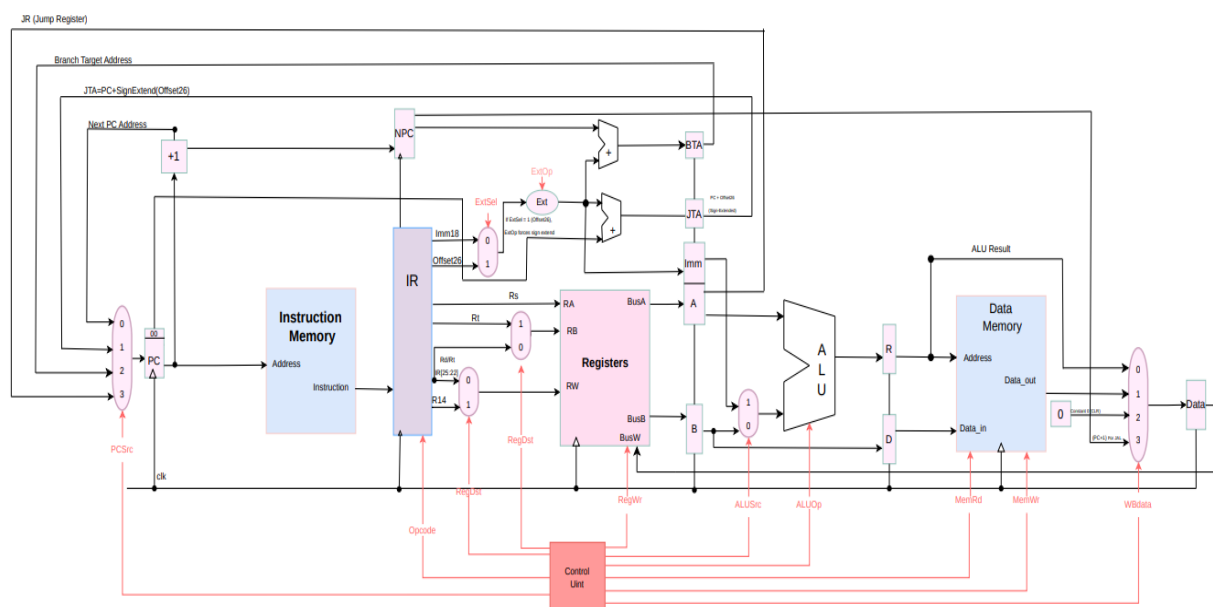


Figure 1: Complete Datapath.

1.3.1 Program Counter (PC) and the PCupdate / PCSrc Logic

The Program Counter is a 32-bit synchronous register, the processor's address bookmark. Every instruction begins here: the PC's current value is sent to Instruction Memory as a word address, selecting which 32-bit instruction to fetch. After the fetch, the PC must advance to the next instruction. But it must *not* advance in ID, EX, MEM, or WB, because the processor is still executing the instruction it already fetched. An always-on increment would race ahead and overwrite the address before the current instruction finishes. The solution is an enable signal called pcupdate (also written pc_update in *processor.v*). When pcupdate is low, the PC register is frozen. When it is high, the PC loads a new value from the 4-input PC MUX on the next rising clock edge.

The pcupdate expression is: $IRWrite \mid PCSrc[0] \mid PCSrc[1]$. This one-line formula deserves careful attention because it encodes all four scenarios in which the PC legitimately changes. First, IRWrite is asserted only in the IF stage, so the PC always advances to pc+1 at the end of every IF stage, for every instruction, no exceptions. Second, if PCSrc is anything other than 00 (meaning a branch, jump, or JR is in progress), at least one of PCSrc[0] or PCSrc[1] is 1, so pcupdate fires and the PC loads the branch or jump target. In every other stage, ID without a jump, EX without a branch, MEM, WB, both IRWrite and PCSrc are 00, so pcupdate is 0 and the PC is frozen. The formula is both minimal and complete.

A subtle hardware note: PC.v uses an active-LOW reset. In the module, the reset port fires when it detects a *falling* edge (negedge). The system-level reset in *processor.v* is active-HIGH (rst). The top-level module inverts it when connecting: it passes `reset(~rst)` to the PC module. This means when the system asserts `rst=1`, the PC module sees 0 on its reset port — which is exactly the active-LOW condition it needs to reset pc to zero. This is a common design pattern when mixing modules with different reset polarities.

The PC MUX 4 Inputs Selected by PCSrc[1:0]

Four completely different sources can become the next PC, depending on what instruction is executing and whether a branch is taken. The 2-bit PCSrc signal selects among them. Understanding why exactly four sources are needed, and why each encodes the way it does is the key to understanding how control flow works in this processor.

Table 7: PC MUX Selection Logic

PCSrc	Source Expression	Meaning	Instruction(s)	Stage Asserted
00	pc + 1	Sequential execution — the default. Every instruction uses this unless something overrides it. The +1 adds one word (one instruction), not one byte, because memory is word-addressable.	all (default)	IF (via IRWrite)
01	pc + Imm32	PC-relative branch. The crucial detail: by the time EX stage runs for a branch instruction, the PC has already been incremented to PC+1 during IF. So the target address = (original_PC + 1) + signed_offset, which equals pc (current) + Imm32. When encoding branch immediates in program.mem, the offset must account for this already-incremented PC.	BEQ, BNE (taken)	EX
10	Imm32	Absolute jump. The sign-extended 26-bit offset from the J-type instruction field is loaded directly as the new PC — it is the absolute target word address, not relative to anything.	J, JAL	ID
11	A register	Register-indirect jump. The pipeline buffer A holds R[rs] (loaded from the Register File during ID). The PC jumps to whatever value was in that register. This allows computed jumps to runtime-determined addresses.	JR	EX

PCSrc=00 is the default selection during the instruction fetch stage, where the PC is updated to PC+1 through the IRWrite term in the pcupdate expression. The other PCSrc values are asserted only during their corresponding execution stages, such as branch, jump, JAL, or JR..

1.3.2 Instruction Memory and the Reason IR Exists

The Instruction Memory is a 256-word, 32-bit-wide, read-only memory. It is loaded at simulation start from *program.mem* using the \$readmemb system task. Its output `instruction_out` is a purely combinational wire: it is always active, takes `pc [7:0]` as an address, and immediately reflects whatever 32-bit word is stored at that location. There is no clock, no enable signal, and no latency. The instant PC changes, `instruction_out` changes.

This is exactly the problem the Instruction Register (IR) was designed to solve. Consider what happens at the end of IF: the FSM transitions to ID, and `pcupdate` fires, loading `pc+1` into the PC register. At that exact same rising edge, the Instruction Memory is still connected to the old PC value so `instruction_out` briefly holds the correct instruction. But in the very next clock cycle (ID), the PC is now `pc+1`. The combinational Instruction Memory immediately output *the next instruction*. If any part of the datapath were reading from `instruction_out` directly in ID, EX, or MEM, it would be reading the wrong instruction, the one at `pc+1`, not the one being executed.

The Instruction Register is a 32-bit synchronous register that latches `instruction_out` at the end of IF (when `IRWrite=1`) and then holds that frozen copy for all remaining stages. `IRWrite` is only ever 1 in the IF state. In ID, EX, MEM, and WB, `IRWrite=0`, so IR does not update, it holds the current instruction steady while the PC races ahead. The decoder, register file, extender, and FSM all read from IR, never from the live `instruction_out` wire. The separation of `instruction_out` (live, changes with PC) from IR (frozen, valid for all stages) is the foundational design decision that makes multicycle execution possible.

1.3.3 Instruction Decoder

The Instruction Decoder has no clock, no logic gates, no state. It is literally a set of wire labels, named slices of the 32-bit IR output. Each field of the instruction format (opcode, rd, rs, rt, immediate, offset) appears at a specific range of bits in IR, and the decoder simply gives those bit ranges readable names so that downstream modules can connect to them cleanly.

Six output wires are produced. The opcode (bits 31:26) goes directly to the FSM, which uses it to decide what state to transition to and which control signals to assert. The rd (bits 25:22), rs (bits 21:18), and rt (bits 17:14) go to the Register File as read addresses. The immediate (bits 17:0, 18 bits) and offset (bits 25:0, 26 bits) go to the Extender.

One encoding subtlety: bits [17:14] of the instruction word simultaneously represent rt (the second source register for R-type and some I-type instructions) and the four most significant bits of the 18-bit immediate field for other I-type instructions. This overlap is resolved by the datapath: the decoder exposes both the register file uses bits [17:14] as a register address to read BusB; the extender uses bits [17:0] as the immediate. Which output actually matters in a given stage is determined by what the FSM asserts: if `BWrite=1`, BusB is used as register data; if `ALUSrc=1`, the extended immediate from the Extender is used instead of BusB.

1.3.4 Register File 16 Registers, Two Read Ports, One Write Port

The Register File contains sixteen 32-bit general-purpose registers (R0 through R15). R0 is hardwired to zero: any attempt to write to it is silently discarded in hardware, and any read of it always returns 0. This is not enforced by a special circuit, it simply follows from the fact that registers [0] is never written, so it retains its reset value of 0 forever.

The two read ports are combinational. There is no clock involved, no enable signal needed, and no wait time. The instant *rs* or *rt* addresses are presented on the read ports, BusA and BusB immediately carry the corresponding register values as wires. This is the correct design for a processor because reading a register carries absolutely no side effect, reading R5 ten thousand times and never using the value causes no harm to any system state. It costs nothing to "waste" a read. Therefore, the register file reads *continuously*, on every cycle, whether or not the current stage actually needs the values. The pipeline buffers A and B are what determine whether those values are *captured and used*.

This reasoning directly explains why there is no RegRead control signal in this design. A RegRead signal would add complexity without providing safety. Compare this to RegWrite, which absolutely must be controlled: accidentally writing the wrong value to a register at the wrong time permanently corrupts processor state and causes incorrect results for all subsequent instructions that read that register. Writes are dangerous; reads are harmless. So only writes get a control signal.

The one write port is fully synchronous. Writes only happen at the rising clock edge, only when RegWrite=1, and only if the destination register is not R0. The clock edge is the "safe commit moment": all the write data and write address signals must settle to correct, stable values *before* the rising edge arrives, and then the edge locks in those stable values atomically. Any glitches that occurred during the clock cycle, transient spikes on the write bus, momentary incorrect values during MUX settling, are completely invisible to the synchronous write, because those glitches are gone before the clock edge samples the inputs.

1.3.5 Buffers A and B Protecting Read Values from Stage Interference

BusA and BusB are combinational outputs of the Register File, permanently driven by the *rs* and *rt* address fields decoded from IR. Because IR is frozen after the IF stage (IRWrite=1 only in IF, never again), the read addresses never change, so BusA and BusB remain stable for the entire duration of the instruction.

Registers A and B exist for a different, simpler reason: they create an explicit registered boundary at the ID→EX transition. Instead of driving the ALU directly from combinational register-file outputs, A and B buffer those values at the ID clock edge (AWrite=1, BWrite=1), giving the ALU stable registered inputs with a full clock period of timing margin through EX.

Register B has an additional role for SW instructions. The store word instruction needs the value of R[rt] to be available all the way in the MEM stage, not to compute an address (that uses R[rs] through A), but to supply the data that gets written into memory. Since B is latched in ID and held through EX and MEM, it naturally carries R[rt] into the MEM stage without any extra wiring or mux needed.

PCsave is a 32-bit register that shares the AWrite enable with register A. When AWrite fires in the ID stage, two things are latched simultaneously: A captures BusA (R[rs]), and PCsave captures the current value of pc. Since PC was already incremented to pc+1 during the IF stage, PCsave ends up holding pc+1 exactly the return address that JAL needs. The reason PCsave shares AWrite instead of having its own enable: AWrite is asserted exclusively in the ID stage, and PCsave only needs to capture the return address in the ID stage. No extra control signal is needed; the timing already aligns perfectly. In WB, when the FSM asserts WBData=10 (PCsave) and RegDst=10 (R14), PCsave's value is written into R14 as the JAL link register value.

The simultaneous clock-edge update for JAL (PCsave latches pc while PC simultaneously jumps to Imm32) is safe because Verilog non-blocking assignments evaluate all right-hand sides *first* using pre-edge values, then write all left-hand sides *simultaneously*. Both PCsave and the PC register read the old value of pc (= original_PC + 1) in their evaluation phase, then write their new values in the update phase. Neither sees the other's write. This is why non-blocking assignments (<=) are used universally for all register updates in this design.

1.3.6 Sign/Zero Extender Making 18-bit and 26-bit Constants ALU-Ready

The ALU always operates on 32-bit values, but the instruction word can only spare 18 bits for an immediate constant (I-type) or 26 bits for a jump offset (J-type). The Extender converts these short values into full 32-bit operands. It is purely combinational, its output Imm32 is a live wire that updates the instant its control inputs (ExtSel and ExtOp) or its data inputs (the immediate/offset fields from IR) change.

Three extension modes cover all 20 instructions, selected by two 1-bit control signals. ExtSel chooses the input source: 0 selects the 18-bit immediate field from IR[17:0], and 1 selects the 26-bit J-type offset from IR[25:0]. ExtOp chooses the extension method: 0 zero-extends (fills the upper bits with 0s), and 1 sign-extends (replicates the most significant bit of the input to fill the upper bits).

Table 8: Extender Mode Selection.

ExtSel	ExtOp	Input	Operation	Why This Choice?	Used By
0	0	IR[17:0] — 18 bits	Zero- extend to 32 bits	ANDI and ORI treat the immediate as a bitmask. Bit 17 being 1 should not "spread" into the upper 14 bits — that would corrupt the mask. Zero-fill preserves the mask exactly.	ANDI, ORI
0	1	IR[17:0] — 18 bits	Sign- extend to 32 bits	Arithmetic, memory offsets, and branch offsets use signed two's-complement integers. Sign-extension preserves the numeric value — a negative offset in 18 bits must remain negative in 32 bits.	ADDI, LW, SW, SWAP, BEQ, BNE
1	×	IR[25:0] — 26 bits	Sign- extend to 32 bits	J and JAL use a 26-bit absolute word address embedded in the instruction. Sign-extending it to 32 bits gives the full target address directly.	J, JAL

A critical timing decision governs *when* ExtOp and ExtSel are asserted. Because Imm32 is a live combinational wire, its value changes the instant ExtOp or ExtSel changes. For I-type instructions (ADDI, ANDI, ORI, LW, SW, SWAP, BEQ, BNE), the ALU reads Imm32 during the EX stage. If ExtOp were asserted in ID, it would be correct during ID, but when the FSM transitions to EX, all outputs reset to their defaults (ExtOp=0, ExtSel=0). The Extender's output would immediately change to zero-extended 18-bit, the wrong value, before the ALU reads it. The solution: assert ExtOp and ExtSel in the *same EX stage where the ALU reads Imm32*. The signal is correct exactly when needed.

For J and JAL, ExtSel is asserted in ID , and this is correct because the jump target ($PC \leftarrow Imm32$) is committed at the ID clock edge. Imm32 only needs to be correct in that one cycle, and J/JAL never reach EX, so there is no risk of it reverting. The rule is elegant: assert ExtSel/ExtOp in the stage that *consumes* Imm32.

1.3.7 ALU-B Multiplexer (ALUSrc) and the ALU

Immediately before the ALU, a 2-input multiplexer selects the second operand. The control signal ALUSrc makes the choice: ALUSrc=0 sends the B pipeline register ($R[rt]$ from the register file), and ALUSrc=1 sends the Imm32 wire from the Extender. The first operand (the A input to the ALU) is always register A , no mux is needed there because the ALU's first operand is always a register value in all 20 instructions of this ISA.

This one-bit signal is the entire difference between "register arithmetic" and "immediate arithmetic." For R-type instructions like ADD and SUB, both operands come from registers (ALUSrc=0). For I-type arithmetic like ADDI or ORI, the second operand is the constant from the instruction (ALUSrc=1). For memory address calculation (LW, SW, SWAP), the address is $R[rs] + \text{sign_ext}(\text{offset})$, so the offset goes through the Extender and ALUSrc=1. For branches (BEQ, BNE), the ALU computes $R[rs] - R[rt]$ to test equality, so both operands come from registers (ALUSrc=0).

The ALU itself is a 32-bit purely combinational unit. It receives two 32-bit inputs (the A register and whatever ALUSrc selected), performs one of nine operations chosen by the 4-bit ALUOp control code, and immediately presents the 32-bit result on its output wire. There is no clock; the output settles within gate propagation delay.

Table 9:ALU Operations.

ALUOp	Operation	Computes	Used by (role)
4'd0 — ADD	Addition	$A + B$	ADD, ADDI (arithmetic); LW, SW, SWAP (address = base + offset)
4'd1 — SUB	Subtraction	$A - B$	SUB (arithmetic); BEQ, BNE (comparison via zero flag)
4'd2 — AND	Bitwise AND	$A \& B$	AND, ANDI

ALUOp	Operation	Computes	Used by (role)
4'd3 — OR	Bitwise OR	$A \mid B$	OR, ORI
4'd4 — XOR	Bitwise XOR	$A \wedge B$	XOR
4'd5 — NOR	Bitwise NOR	$\sim(A \mid B)$	NOR
4'd6 — SLL	Shift left logical	$A \ll B[4:0]$	SLL (5 bits of B used as shift amount; 32-bit max shift = 31 requires 5 bits)
4'd7 — SRL	Shift right logical	$A \gg B[4:0]$	SRL
4'd8 — ZERO	Force zero output	32'd0	CLR: writes zero directly without using A or B — the ALU output is just constant zero

The zero flag is a single combinational wire: it is 1 whenever the ALU result equals zero, and 0 otherwise. It updates instantly as the ALU result changes. For BEQ and BNE, the entire branch decision happens within the EX clock cycle: the FSM sets $ALUOp = SUB$ and $ALUSrc = 0$, the ALU computes $A - B$, the zero flag wire changes, the FSM's combinational output block re-evaluates and conditionally sets $PCSrc$, and the PC MUX presents the correct $next_pc$, all before the rising clock edge arrives to commit the result. No extra "branch delay" stage is required because combinational settling is faster than one clock period.

1.3.8 ALUOut The Indispensable Address Safety Buffer

The ALU result wire is live and combinational, it reflects whatever $ALUSrc$ and $ALUOp$ are currently set to. When the FSM exits EX and enters MEM, the default signal values kick in: $ALUSrc$ resets to 0, $ALUOp$ resets to ALU_ADD . Immediately, the ALU starts computing $A + B$ (whatever is currently in A and B, not the carefully computed memory address from EX), and the live result wire changes to this garbage value. Without a buffer, the Data Memory's address

input would receive this incorrect value, causing the processor to read from or write to the completely wrong memory location a silent, catastrophic bug.

ALUOut is a 32-bit register that latches the correct ALU result at the end of EX (when ALUOutWrite=1). Once latched, ALUOut holds its value regardless of what happens to the live ALU result wire in subsequent stages. In the MEM stage, the Data Memory's address input is connected to ALUOut (the stable buffer), not to the live alu_result wire. The correct address is always there, frozen and safe, while the datapath's live signals do whatever they need to do for the next stage's control setup.

ALUOut also feeds into the WB Data MUX as the source for write-back when WBData=00 (R-type and I-type ALU results). For these instructions, the result computed in EX must survive into WB without being corrupted by the stage transitions in between. ALUOut provides this stable bridge.

1.3.9 Data Memory The Harvard-Separated R/W Store

Data Memory is a 256-word, 32-bit-wide, read/write memory. It is completely separate from Instruction Memory, this is the Harvard architecture. The two memories have independent address and data buses, which means an instruction fetch and a data access could theoretically happen in the same cycle without a structural hazard. In this multicycle design that concurrency is not used, but the architectural separation is maintained: the processor never fetches data from Instruction Memory, and never executes code from Data Memory.

The address input to Data Memory is permanently connected to ALUOut. This is correct: the effective address for any memory operation is always an ALU result (base register + sign-extended offset). The MemRd signal gates the read: when MemRd=1, the memory's output wire (mem_data_out) immediately presents the 32-bit word at the ALUOut address, no clock needed, purely combinational. When MemRd=0, the output is don't-care. The MemWr signal gates the write: when MemWr=1, the value from register B (which holds R[rt] for SW, the value to be stored) is written into memory at ALUOut's address at the rising clock edge. The synchronous write protects against glitches exactly as it does in the Register File only the stable, settled value at the clock edge is committed.

1.3.10 MDR (Memory Data Register) Protecting the Loaded Value into WB

mem_data_out is a combinational wire from Data Memory. When the MEM stage ends and the processor transitions to WB, the FSM defaults MemRd back to 0. At that moment, the memory's output wire becomes undefined the memory no longer knows what address to respond to (or does not care, since it is being told not to read). Without a buffer, the WB stage would try to write this undefined value into the Register File, corrupting R[rd] with garbage data.

MDR (Memory Data Register) is a 32-bit register that latches mem_data_out at the end of MEM (when MDRWrite=1). MDRWrite=1 only for LW and SWAP instructions during the MEM stage. Once latched, MDR holds the loaded word stably through the transition into WB, where the FSM

asserts `WBData=01` to route MDR's value through the Write-Back MUX into the Register File write port. MDR solves exactly the same problem ALUOut solves, one stage later: a live combinational signal that must be frozen before the next stage can safely use it.

1.3.11 The Memories Compared Side by Side

Table 10: Memory Comparison.

Property	Instruction Memory	Data Memory
Depth $\times$ Width	256 $\times$ 32-bit words	256 $\times$ 32-bit words
Address input source	pc[7:0]	ALUOut[7:0]
Read behaviour	Always-on combinational read (no enable)	Combinational when MemRd=1; don't-care otherwise
Write behaviour	Not writable during simulation (ROM)	Synchronous write on posedge clk when MemWr=1
Initialisation	\$readmemb("program.mem") at simulation start	All zeros at reset
Active in which stage	IF only	MEM only (LW, SW, SWAP)
Output register needed?	Yes — IR latches instruction_out	Yes — MDR latches mem_data_out

Both instruction and data memories require output registers, IR and MDR respectively, because their combinational outputs may become invalid once the corresponding memory control signals change.

1.3.12 The Four Multiplexers Where Control Signals Meet Data Paths

Multiplexers are the points in the datapath where the FSM's decisions become physical data routing. Every MUX in this processor exists because at least two different instructions need to send different data to the same downstream hardware at different times. Rather than duplicating hardware, the MUX selects the right source based on a control signal. All four MUXes in this design are implemented as combinational assign statements, they have zero latency and zero hardware cost beyond wiring and gate logic.

Table 11: Multiplexer Summary..

MUX	Select	Input 0 (select=0)	Input 1 (select=1)	Input 2 (10)	Input 3 (11)	Why these inputs needed
PC MUX (4:1)	PCSrc[1:0]	pc+1 (sequential)	pc+Imm32 (branch)	Imm32 (jump)	A register (JR)	The processor has four fundamentally different ways to determine the next instruction address — one for each control-flow mode. No two can be collapsed.
ALU-B MUX (2:1)	ALUSrc	B register (R[rt])	Imm32 from Extender	—		R-type instructions use a register as the second operand; I-type use a constant. Two sources, one ALU input port.

MUX	Select	Input 0 (select=0)	Input 1 (select=1)	Input 2 (10)	Input 3 (11)	Why these inputs needed
WB Data MUX (4:1)	WBData[1:0]	ALUOut (R/I ALU)	MDR (load)	PCsave (JAL)	32'd0 (CLR)	Four distinct data sources can be written back to the Register File. CLR writes hardwired zero (no ALU needed). JAL writes the return address. Load writes memory data. Everything else writes the ALU result.
RegDst MUX	RegDst[1:0]	rd field from IR (normal destination register)		R14 (JAL link register)	—	Every instruction writes to the rd field — except JAL, which must always write to R14 (the hardcoded link register). The MUX lets JAL override the destination without changing the instruction encoding.

The control unit is a Moore FSM: outputs depend only on the current state, not directly on the opcode or zero inputs. This prevents instruction-dependent glitches from reaching the datapath between clock edges. The module uses the standard three-process FSM template, each always block with a precisely defined sensitivity list.

An important Verilog nuance: all control outputs are declared output reg but are driven by always @(*), they are therefore combinational logic, not hardware registers. Verilog requires the reg keyword for any variable assigned inside an always block, regardless of whether flip-flop storage results. Only State, driven by always @(posedge clk), is a real sequential element.

1.4 Control Unit FSM Design

1.4.1 FSM Architecture

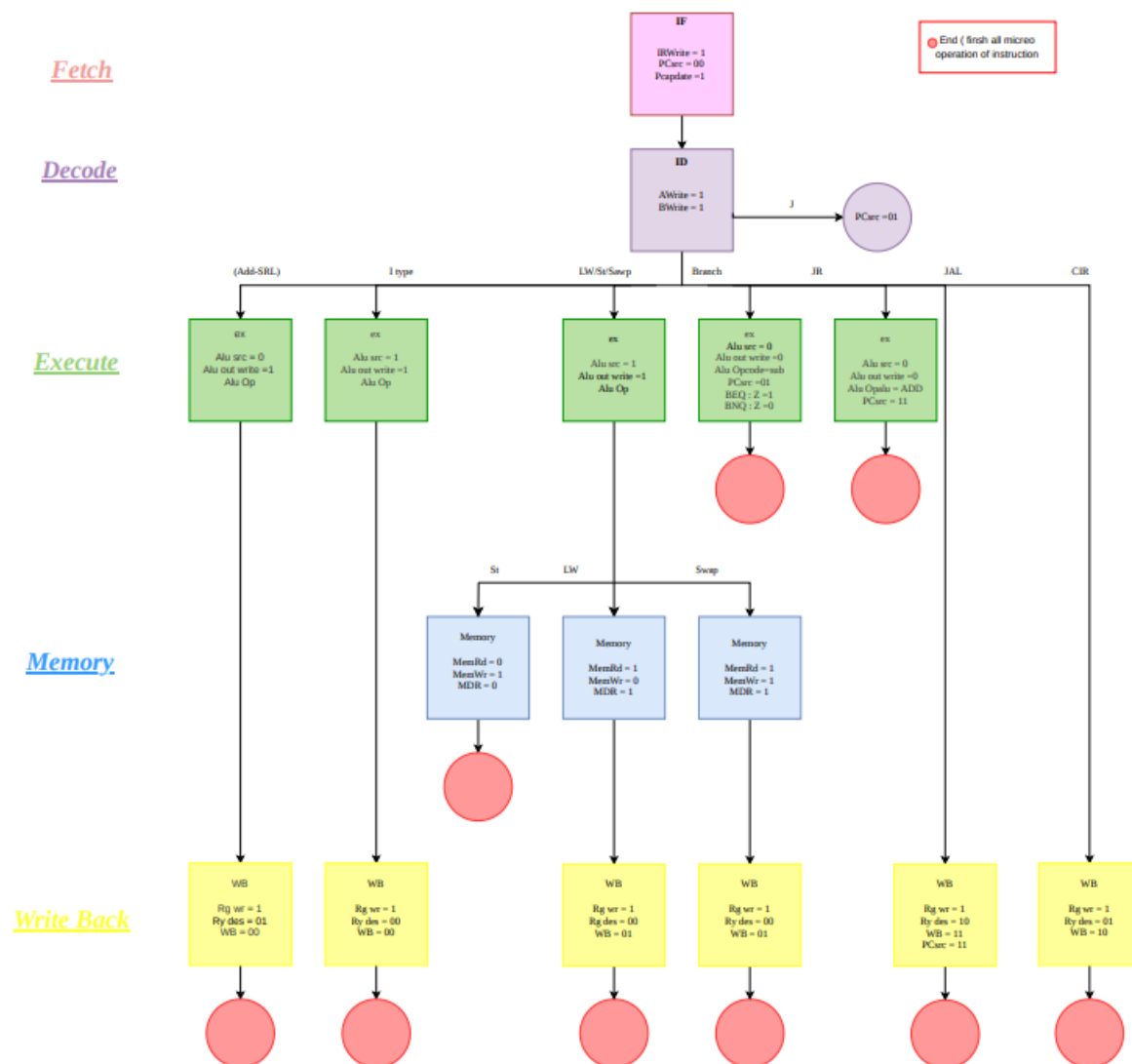


Figure 2:FSM State Diagram

1.4.2 Next-State Transition Table

Table 12 : FSM Next-State Transition Table

	Condition	Next State	Rationale
IF (0)	always	ID	Every instruction requires decode stage
ID (1)	J (opcode 18)	IF	PC loaded in ID; instruction complete in 2 cycles
	CLR (8), JAL (19)	WB	CLR writes 0 directly; JAL writes PCsave — no ALU computation needed
	all others	EX	ALU operation or address calculation required
EX (2)	LW (13), SW (14), SWAP (15)	MEM	Memory read or write required
	BEQ (16), BNE (17), JR (9)	IF	PC updated in EX; no register writeback
	all R-type ALU, I-type ALU	WB	ALUOut result must be written to register file
MEM(3)	SW (14)	IF	Store has no register destination; skip WB
	LW (13), SWAP (15)	WB	MDR data must be committed to register file
WB (4)	always	IF	Instruction complete; fetch next

1.4.3 Output Logic Complete Control Signal Table

Table 13: Complete FSM Output Logic.

State	Instruction(s)	IRWr	AWr	BWr	AOWr	MDWRr	RegWr	MRd	MWr	ExtOp	ExtSel	ASrc	ALUOp	PCSrc	WBDat	RgDst
IF	all	1	0	0	0	0	0	0	0	0	0	0	ADD	00	00	00
ID	All (non-J/JAL)	0	1	1	0	0	0	0	0	0	0	0	—	00	00	00
ID	J, JAL	0	1	1	0	0	0	0	0	0	1	0	—	10	00	00
EX	ADD	0	0	0	1	0	0	0	0	0	0	0	ADD	00	00	00
EX	SUB	0	0	0	1	0	0	0	0	0	0	0	SUB	00	00	00
EX	AND	0	0	0	1	0	0	0	0	0	0	0	AND	00	00	00
EX	OR	0	0	0	1	0	0	0	0	0	0	0	OR	00	00	00
EX	XOR	0	0	0	1	0	0	0	0	0	0	0	XOR	00	00	00
EX	NOR	0	0	0	1	0	0	0	0	0	0	0	NOR	00	00	00
EX	SLL	0	0	0	1	0	0	0	0	0	0	0	SLL	00	00	00
EX	SRL	0	0	0	1	0	0	0	0	0	0	0	SRL	00	00	00
EX	ADDI	0	0	0	1	0	0	0	0	1	0	1	ADD	00	00	00
EX	ANDI	0	0	0	1	0	0	0	0	0	0	1	AND	00	00	00
EX	ORI	0	0	0	1	0	0	0	0	0	0	1	OR	00	00	00
EX	LW, SW, SWAP	0	0	0	1	0	0	0	0	1	0	1	ADD	00	00	00
EX	BEQ (taken)	0	0	0	0	0	0	0	0	1	0	0	SUB	01	00	00
EX	BNE (taken)	0	0	0	0	0	0	0	0	1	0	0	SUB	01	00	00
EX	JR	0	0	0	0	0	0	0	0	0	0	0	—	11	00	00
MEM	LW, SWAP (read)	0	0	0	0	1	0	1	0	0	0	0	—	00	00	00
MEM	SW (write)	0	0	0	0	0	0	0	1	0	0	0	—	00	00	00
MEM	SWAP (write)	0	0	0	0	1	0	1	1	0	0	0	—	00	00	00
WB	R-type / I- ALU	0	0	0	0	0	1	0	0	0	0	0	—	00	00	01
WB	LW, SWAP	0	0	0	0	0	1	0	0	0	0	0	—	00	01	00
WB	CLR	0	0	0	0	0	1	0	0	0	0	0	—	00	11	01

1.5 Boolean Equations of Key Control Signals

The following equations are derived directly from the FSM output logic in *C.v*. All outputs default to 0; each equation describes only the conditions under which a signal is asserted (= 1) or takes a non-zero value.

1.5.1 PC Update Enable

The PC register has an implicit write-enable: it only updates when the control unit explicitly drives it. In the Verilog implementation(*PC.v*)the PC updates on any rising clock edge while PCupdate=1:

```
// Derived from FSM: PC is written in IF (IRWrite path) and whenever PCSrc ≠ 00
PCupdate = IRWrite | PCSrc[1] | PCSrc[0]
```

Figure 3 :PC Update Enable Logic

In words: the PC updates during **IF** (where $IRWrite = 1$ and $PCSrc = 00$ loads $PC+4$), and during any state that resolves a branch or jump ($PCSrc \neq 00$). In all other states the PC holds its value.

1.5.2 PCSrc Multiplexer Select

$PCSrc$ is a 2-bit select driven by the FSM. The four cases map as follows:

Table 14 : $PCSrc$ encoding and source selection.

$PCSrc[1:0]$	$PC \leftarrow$	Condition
00	$PC + 4$	State = IF (normal increment)
01	ALUOut	State = EX, Opcode = BEQ/BNE and branch taken
10	Imm32 (sign-extended)	State = ID, Opcode = J or JAL
11	R[rs] (register)	State = EX, Opcode = JR

1.5.3 Branch-Taken Logic

Branches are resolved combinatorially during the EX stage. The ALU computes $A - B$ and a zero flag is raised if the result is all zeroes.

```

zero = (ALU_result == 32'd0)

// BEQ: branch if equal (zero = 1)
BEQ_taken = (State == EX) & (Opcode == BEQ) & zero

// BNE: branch if not equal (zero = 0)
BNE_taken = (State == EX) & (Opcode == BNE) & ~zero

// FSM assertion: if taken → PCSrc = 01 (ALUOut holds branch target)
PCSrc = (BEQ_taken | BNE_taken) ? 2'b01 : 2'b00

```

Figure 4 :Branch Decision and $PCSrc$ Control Logic.

Because the FSM is Moore, these outputs settle combinatorially *during* EX and are sampled on the next rising edge, so no extra delay cycle is required.

1.5.4 State-Decoded Single-Bit Signal Equations

The single-bit outputs that are high for exactly one state reduce to direct state comparisons:

```
IRWrite = (State == IF)

AWrite = (State == ID)

BWrite = (State == ID)

ALUOutWrite = (State == EX)

MDRWrite = (State == MEM) & (Opcode != SW)

MemRd = (State == MEM) & (Opcode == LW | Opcode == SWAP)

MemWr = (State == MEM) & (Opcode == SW | Opcode == SWAP)

RegWr = (State == WB) | (State == ID & (Opcode == CLR | Opcode == JAL))
```

Figure 5 : Complete FSM Output Logic.

1.5.5 State-Decoded Single-Bit Signal Equations

WBData selects the value written to the register file; RegDst selects the destination register:

Table 15:WBData encoding.

WBData[1:0]	Register ←	Used by
00	ALUOut	ADD, SUB, AND, OR, ADDI, ANDI, ORI, SLL, SRL, SRA, CLR, NOT, MOV, SWAP
01	MDR	LW
10	PCsave	JAL (R14 ← return address)
11	— (unused)	—

Table 16 : RegDst encoding.

RegDst[1:0]	Destination Register	Used by
00	IR[22:19] (rt field)	ADDI, ANDI, ORI, LW, SWAP
01	IR[18:15] (rd field)	R-type (ADD, SUB, AND, OR, SLL, SRL, SRA, NOT, MOV)
10	R14 (hardwired)	JAL (link register)
11	R0 (hardwired zero)	CLR (IR[22:19] forced to R0 write ignored)

1.5.6 ALUSrc and ALUOp Equations

```
// ALUSrc = 1: second ALU operand comes from Imm32 (not from register B)
ALUSrc = (State == EX) & (Opcode == ADDI | Opcode == ANDI | Opcode == ORI
    | Opcode == LW | Opcode == SW | Opcode == SWAP
    | Opcode == BEQ | Opcode == BNE)

// ALUOp: encoded per-opcode during EX (default = ALU_ADD = 0 for address calc)
ALUOp = (State == EX) ? opcode_to_aluop(Opcode) : ALU_ADD
```

Figure 6: ALUSrc and ALUOp Control Equations

Shift-amount instructions (SLL, SRL, SRA) drive ALUSrc = 0; the shift amount is carried in IR [14:10] and routed directly by the ALU module when the corresponding ALUOp is set.

2. Testing and Simulation

To verify the multicycle RISC processor, a complete test program was written in program.mem. The program includes R-Type instructions, I-Type ALU instructions, memory instructions, branch instructions, and jump instructions. The goal of the test is to make sure that the datapath, control unit, register file, ALU, memory system, PC update logic, and write-back paths are working correctly.

Test Program:

The testing is divided into three main parts. The first part tests the R-Type instructions, including arithmetic, logic, shift, CLR, and JR instructions. The second part tests the I-Type, memory, and branch instructions, including ADDI, ANDI, ORI, LW, SW, SWAP, BEQ, and BNE. The third part tests the J-Type instructions, including J and JAL. At the end of each part, the final result explains whether the tested instructions executed correctly and which processor components were verified.

Initial Register and Memory Values:

Before running the simulation, the testbench initializes the needed registers and memory locations. Register R1 is initialized to 10, and R2 is initialized to 3. These two registers are used as source operands for most arithmetic and logic instructions. Register R12 is initialized to 11 so that the JR instruction can jump to address 11. Register R13 is initialized to 20 and is used as the base address for memory instructions.

The data memory is also initialized for the memory instructions. Mem[22] is initialized to 99 for the LW instruction. Mem[23] is initialized to 0 for the SW instruction. Mem[24] is initialized to 88 for the SWAP instruction.

2.1 R-Type Instructions Test

The following table shows the R-Type instruction sequence used in the test program. Each row contains the instruction address, the 32-bit binary machine code, the decoded assembly instruction, and the operation performed by the processor.

Table 17 :R Type Instructions program

Address	Binary Instruction	Instruction	Meaning
0	00000000110001001000000000000000	ADD R3, R1, R2	$R3 = R1 + R2$
1	00000101000001001000000000000000	SUB R4, R1, R2	$R4 = R1 - R2$
2	00001001010001001000000000000000	AND R5, R1, R2	$R5 = R1 \& R2$
3	00001101100001001000000000000000	OR R6, R1, R2	$R6 = R1 R2$
4	00010001110001001000000000000000	XOR R7, R1, R2	$R7 = R1 \wedge R2$
5	00010110000001001000000000000000	NOR R8, R1, R2	$R8 = \sim(R1 R2)$
6	00011010010010001000000000000000	SLL R9, R2, R2	$R9 = R2 \ll R2$
7	00011110100001001000000000000000	SRL R10, R1, R2	$R10 = R1 \gg R2$
8	00100010110000000000000000000000	CLR R11	$R11 = 0$
9	00100100001100000000000000000000	JR R12	$PC = R12$
10	00100000110000000000000000000000	CLR R3	$R3 = 0$
11	00000011110011000000000000000000	ADD R15, R3, R0	$R15 = R3 + R0$

Before running the instruction sequence, the testbench initializes some registers with specific values. In this test, $R1=10$, $R2=3$, $R11=55$, and $R12=11$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 1 : ADD R3, R1, R2
◦ # KERNEL: =====
◦ # KERNEL: Cycle00 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=0 | IR=00000000000000000000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle01 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
◦ # KERNEL:   Pipeline Registers: IR=00000000110001001000000000000000 | A=0 | B=0
◦ # KERNEL:
◦ # KERNEL: Cycle02 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=0 | ALUOutWrite=1
◦ # KERNEL:   Pipeline Registers: A=10 | B=3 | ALUOut=0
◦ # KERNEL:
◦ # KERNEL: Cycle03 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=01 | WBDData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=13 | MDR=0 | WriteReg=R3 | BusW=13
◦ # KERNEL:

```

Figure 7 :ADD R3, R1, R2

IF: The processor fetches the instruction at PC=0. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially.

ID: The instruction stored in IR is decoded as ADD R3, R1, R2. The control signals AWrite=1 and BWrite=1 enable storing the register file outputs into A and B. In the printed output, A=0 and B=0 still appear because the new values are latched at the end of the ID cycle.

EX: the values that were read in the previous ID cycle are now available in the pipeline registers. Register A contains the value of R1, which is 10, and register B contains the value of R2, which is 3. Since ALUSrc=0, the ALU uses register B as its second input instead of an immediate value. The control signal ALUOp=0 selects the ADD operation. Therefore, the ALU computes $10 + 3 = 13$. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed value ALUOut=0 is the old value before the new result is latched.

WB: the ALU result is now stored in ALUOut, and its value is 13. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R3, and since WBDData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 13 into register R3. This confirms that the instruction ADD R3, R1, R2 was executed correctly, because $R3 = R1 + R2 = 10 + 3 = 13$

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 2 : SUB R4, R1, R2
◦ # KERNEL: =====
◦ # KERNEL: Cycle04 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=1 | IR=00000000110001001000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle05 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
◦ # KERNEL:   Pipeline Registers: IR=00000101000001001000000000000000 | A=10 | B=3
◦ # KERNEL:
◦ # KERNEL: Cycle06 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=1 | ALUOutWrite=1
◦ # KERNEL:   Pipeline Registers: A=10 | B=3 | ALUOut=13
◦ # KERNEL:
◦ # KERNEL: Cycle07 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=01 | WBDData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=7 | MDR=0 | WriteReg=R4 | BusW=7
◦ # KERNEL:

```

Figure 8 : SUB R4, R1, R2

IF: The processor fetches the instruction at PC=1. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as SUB R4, R1, R2. The control signals AWrite=1 and BWrite=1 enable storing the register file outputs into A and B. In this instruction, R1 is the first source register and R2 is the second source register, so A receives the value of R1, which is 10, and B receives the value of R2, which is 3.

EX: The values read in the previous ID cycle are now available in the pipeline registers. Register A contains the value of R1, which is 10, and register B contains the value of R2, which is 3. Since ALUSrc=0, the ALU uses register B as its second input instead of an immediate value. The control signal ALUOp=1 selects the SUB operation. Therefore, the ALU computes $10 - 3 = 7$. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed value ALUOut=13 is the old value from the previous ADD instruction before the new SUB result is latched.

WB: The ALU result is now stored in ALUOut, and its value is 7. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R4, and since WBDData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 7 into register R4. This confirms that the instruction SUB R4, R1, R2 was executed correctly, because $R4 = R1 - R2 = 10 - 3 = 7$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 3 : AND R5, R1, R2
◦ # KERNEL: =====
◦ # KERNEL: Cycle08 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=2 | IR=00000101000001001000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle09 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
◦ # KERNEL:   Pipeline Registers: IR=00001001010001001000000000000000 | A=10 | B=3
◦ # KERNEL:
◦ # KERNEL: Cycle10 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=2 | ALUOutWrite=1
◦ # KERNEL:   Pipeline Registers: A=10 | B=3 | ALUOut=7
◦ # KERNEL:
◦ # KERNEL: Cycle11 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=01 | WBData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=2 | MDR=0 | WriteReg=R5 | BusW=2
◦ # KERNEL:

```

Figure 9 : AND R5, R1, R2

IF: The processor fetches the instruction at PC=2. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as AND R5, R1, R2. The control signals AWrite=1 and BWrite=1 enable storing the register file outputs into A and B. In this instruction, R1 is the first source register and R2 is the second source register, so A receives the value of R1, which is 10, and B receives the value of R2, which is 3.

EX: The values read in the previous ID cycle are now available in the pipeline registers. Register A contains the value of R1, which is 10, and register B contains the value of R2, which is 3. Since ALUSrc=0, the ALU uses register B as its second input instead of an immediate value. The control signal ALUOp=2 selects the AND operation. Therefore, the ALU computes 10 AND 3. In binary, 10 = 1010 and 3 = 0011, so the result is 0010, which equals 2. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed value ALUOut=7 is the old value from the previous SUB instruction before the new AND result is latched.

WB: The ALU result is now stored in ALUOut, and its value is 2. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R5, and since WBData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 2 into register R5. This confirms that the instruction AND R5, R1, R2 was executed correctly, because $R5 = R1 \text{ AND } R2 = 10 \text{ AND } 3 = 2$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 4 : OR R6, R1, R2
◦ # KERNEL: =====
◦ # KERNEL: Cycle12 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=3 | IR=00001001010001001000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle13 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
◦ # KERNEL:   Pipeline Registers: IR=00001101100001001000000000000000 | A=10 | B=3
◦ # KERNEL:
◦ # KERNEL: Cycle14 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=3 | ALUOutWrite=1
◦ # KERNEL:   Pipeline Registers: A=10 | B=3 | ALUOut=2
◦ # KERNEL:
◦ # KERNEL: Cycle15 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=01 | WBDData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=11 | MDR=0 | WriteReg=R6 | BusW=11
◦ # KERNEL:

```

Figure 10 : OR R6, R1, R2

IF: The processor fetches the instruction at PC=3. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as OR R6, R1, R2. The control signals AWrite=1 and BWrite=1 enable storing the register file outputs into A and B. In this instruction, R1 is the first source register and R2 is the second source register, so A receives the value of R1, which is 10, and B receives the value of R2, which is 3.

EX: The values read in the previous ID cycle are now available in the pipeline registers. Register A contains the value of R1, which is 10, and register B contains the value of R2, which is 3. Since ALUSrc=0, the ALU uses register B as its second input instead of an immediate value. The control signal ALUOp=3 selects the OR operation. Therefore, the ALU computes 10 OR 3. In binary, 10 = 1010 and 3 = 0011, so the result is 1011, which equals 11. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed value ALUOut=2 is the old value from the previous AND instruction before the new OR result is latched.

WB: The ALU result is now stored in ALUOut, and its value is 11. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R6, and since WBDData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 11 into register R6. This confirms that the instruction OR R6, R1, R2 was executed correctly, because $R6 = R1 \text{ OR } R2 = 10 \text{ OR } 3 = 11$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 5 : XOR R7, R1, R2
◦ # KERNEL: =====
◦ # KERNEL: Cycle16 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=4 | IR=00001101100001001000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle17 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
◦ # KERNEL:   Pipeline Registers: IR=00010001110001001000000000000000 | A=10 | B=3
◦ # KERNEL:
◦ # KERNEL: Cycle18 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=4 | ALUOutWrite=1
◦ # KERNEL:   Pipeline Registers: A=10 | B=3 | ALUOut=11
◦ # KERNEL:
◦ # KERNEL: Cycle19 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=01 | WBDData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=9 | MDR=0 | WriteReg=R7 | BusW=9
◦ # KERNEL:

```

Figure 11 : XOR R7, R1, R2

IF: The processor fetches the instruction at PC=4. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as XOR R7, R1, R2. The control signals AWrite=1 and BWrite=1 enable storing the register file outputs into A and B. In this instruction, R1 is the first source register and R2 is the second source register, so A receives the value of R1, which is 10, and B receives the value of R2, which is 3.

EX: The values read in the previous ID cycle are now available in the pipeline registers. Register A contains the value of R1, which is 10, and register B contains the value of R2, which is 3. Since ALUSrc=0, the ALU uses register B as its second input instead of an immediate value. The control signal ALUOp=4 selects the XOR operation. Therefore, the ALU computes 10 XOR 3. In binary, 10 = 1010 and 3 = 0011, so the result is 1001, which equals 9. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed value ALUOut=11 is the old value from the previous OR instruction before the new XOR result is latched.

WB: The ALU result is now stored in ALUOut, and its value is 9. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R7, and since WBDData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 9 into register R7. This confirms that the instruction XOR R7, R1, R2 was executed correctly, because $R7 = R1 \text{ XOR } R2 = 10 \text{ XOR } 3 = 9$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 6 : NOR R8, R1, R2
◦ # KERNEL: =====
◦ # KERNEL: Cycle20 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=5 | IR=00010001110001001000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle21 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
◦ # KERNEL:   Pipeline Registers: IR=00010110000001001000000000000000 | A=10 | B=3
◦ # KERNEL:
◦ # KERNEL: Cycle22 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=5 | ALUOutWrite=1
◦ # KERNEL:   Pipeline Registers: A=10 | B=3 | ALUOut=9
◦ # KERNEL:
◦ # KERNEL: Cycle23 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=01 | WBDData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=-12 | MDR=0 | WriteReg=R8 | BusW=-12
◦ # KERNEL:

```

Figure 12 : NOR R8, R1, R2

IF: The processor fetches the instruction at PC=5. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as NOR R8, R1, R2. The control signals AWrite=1 and BWrite=1 enable storing the register file outputs into A and B. In this instruction, R1 is the first source register and R2 is the second source register, so A receives the value of R1, which is 10, and B receives the value of R2, which is 3.

EX: The values read in the previous ID cycle are now available in the pipeline registers. Register A contains the value of R1, which is 10, and register B contains the value of R2, which is 3. Since ALUSrc=0, the ALU uses register B as its second input instead of an immediate value. The control signal ALUOp=5 selects the NOR operation. Therefore, the ALU computes NOT(10 OR 3). In binary, 10 OR 3 = 1011, and applying NOT gives the two's complement result -12. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed value ALUOut=9 is the old value from the previous XOR instruction before the new NOR result is latched.

WB: The ALU result is now stored in ALUOut, and its value is -12. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R8, and since WBDData=00, the data written back comes from ALUOut. Therefore, the processor writes the value -12 into register R8. This confirms that the instruction NOR R8, R1, R2 was executed correctly, because $R8 = \text{NOT}(R1 \text{ OR } R2) = \text{NOT}(10 \text{ OR } 3) = -12$.

```

° # KERNEL: =====
° # KERNEL: Instruction 7 : SLL R9, R2, R2
° # KERNEL: =====
° # KERNEL: Cycle24 : stage:IF
° # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
° # KERNEL:   Pipeline Registers: PC=6 | IR=00010110000001001000000000000000
° # KERNEL:
° # KERNEL: Cycle25 : stage:ID
° # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
° # KERNEL:   Pipeline Registers: IR=00011010010010001000000000000000 | A=10 | B=3
° # KERNEL:
° # KERNEL: Cycle26 : stage:EX
° # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=6 | ALUOutWrite=1
° # KERNEL:   Pipeline Registers: A=3 | B=3 | ALUOut=-12
° # KERNEL:
° # KERNEL: Cycle27 : stage:WB
° # KERNEL:   Control Signals   : RegDst=01 | WBDData=00 | RegWr=1
° # KERNEL:   Pipeline Registers: ALUOut=24 | MDR=0 | WriteReg=R9 | BusW=24
° # KERNEL:

```

Figure 13 : SLL R9, R2, R2

IF: The processor fetches the instruction at PC=6. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as SLL R9, R2, R2. The control signals AWrite=1 and BWrite=1 enable storing the register file outputs into A and B. In this instruction, both source fields use R2, so both A and B receive the value of R2, which is 3. In the printed output during ID, A=10 and B=3 still appear because these are the old values before the new values are latched at the end of the ID cycle.

EX: The values read in the previous ID cycle are now available in the pipeline registers. Register A contains the value of R2, which is 3, and register B also contains the value of R2, which is 3. Since ALUSrc=0, the ALU uses register B as its second input instead of an immediate value. The control signal ALUOp=6 selects the SLL operation. Therefore, the ALU computes $3 \ll 3$, which means shifting the value 3 left by 3 bits. In binary, $3 = 0011$, and shifting left by 3 gives 11000, which equals 24. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed value ALUOut=-12 is the old value from the previous NOR instruction before the new SLL result is latched.

WB: The ALU result is now stored in ALUOut, and its value is 24. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R9, and since WBDData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 24 into register R9. This confirms that the instruction SLL R9, R2, R2 was executed correctly, because $R9 = R2 \ll R2 = 3 \ll 3 = 24$.

```

◦ # KERNEL : =====
◦ # KERNEL: Instruction 8 : SRL R10, R1, R2
◦ # KERNEL : =====
◦ # KERNEL: Cycle28 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=7 | IR=00011010010010001000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle29 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
◦ # KERNEL:   Pipeline Registers: IR=00011110100001001000000000000000 | A=3 | B=3
◦ # KERNEL:
◦ # KERNEL: Cycle30 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=7 | ALUOutWrite=1
◦ # KERNEL:   Pipeline Registers: A=10 | B=3 | ALUOut=24
◦ # KERNEL:
◦ # KERNEL: Cycle31 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=01 | WBDData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=1 | MDR=0 | WriteReg=R10 | BusW=1
◦ # KERNEL:

```

Figure 14 : SRL R10, R1, R2

IF: The processor fetches the instruction at PC=7. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as SRL R10, R1, R2. The control signals AWrite=1 and BWrite=1 enable storing the register file outputs into A and B. In this instruction, R1 is the value to be shifted, and R2 is the shift amount. Therefore, A receives the value of R1, which is 10, and B receives the value of R2, which is 3. In the printed output during ID, A=3 and B=3 still appear because these are the old values from the previous SLL instruction before the new values are latched at the end of the ID cycle.

EX: The values read in the previous ID cycle are now available in the pipeline registers. Register A contains the value of R1, which is 10, and register B contains the value of R2, which is 3. Since ALUSrc=0, the ALU uses register B as its second input instead of an immediate value. The control signal ALUOp=7 selects the SRL operation. Therefore, the ALU computes $10 \gg 3$, which means shifting the value 10 right by 3 bits. In binary, $10 = 1010$, and shifting right by 3 gives 0001, which equals 1. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed value ALUOut=24 is the old value from the previous SLL instruction before the new SRL result is latched.

WB: The ALU result is now stored in ALUOut, and its value is 1. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R10, and since WBDData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 1 into register R10. This confirms that the instruction SRL R10, R1, R2 was executed correctly, because $R10 = R1 \gg R2 = 10 \gg 3 = 1$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 9 : CLR R11
◦ # KERNEL: =====
◦ # KERNEL: Cycle32 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=8 | IR=00011110100001001000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle33 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
◦ # KERNEL:   Pipeline Registers: IR=00100010110000000000000000000000 | A=10 | B=3
◦ # KERNEL:
◦ # KERNEL: Cycle34 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=01 | WBDData=11 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=1 | MDR=0 | WriteReg=R11 | BusW=0
◦ # KERNEL:

```

Figure 15 : CLR R11

IF: The processor fetches the instruction at PC=8. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as CLR R11. Before executing this instruction, register R11 was initialized in the testbench with the value 55. This value is used only to verify that the CLR instruction works correctly. The control signals AWrite=1 and BWrite=1 are asserted, so the processor still reads from the register file and updates the temporary registers A and B. However, for the CLR instruction, the old value of R11 is not sent to the ALU. The purpose of the instruction is simply to clear the destination register by writing zero to it.

WB: The CLR instruction goes directly to the write-back cycle without an execute cycle. The control signal RegWr=1 enables writing to the register file. Since RegDst=01, the destination register is selected as R11. Since WBDData=11, the write-back data is selected as a constant zero instead of ALUOut or MDR. Therefore, BusW=0, and the processor writes 0 into R11. This confirms that the instruction CLR R11 executed correctly, because R11 was initially 55 and after the write-back stage it became 0.

```

° # KERNEL: =====
° # KERNEL: Instruction 10 : JR R12
° # KERNEL: =====
° # KERNEL: Cycle35 : stage:IF
° # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
° # KERNEL:   Pipeline Registers: PC=9 | IR=00100010110000000000000000000000
° # KERNEL: =====
° # KERNEL: Cycle36 : stage:ID
° # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
° # KERNEL:   Pipeline Registers: IR=00100100001100000000000000000000 | A=0 | B=0
° # KERNEL: =====
° # KERNEL: Cycle37 : stage:EX
° # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=0 | ALUOutWrite=0
° # KERNEL:   Pipeline Registers: A=11 | B=0 | ALUOut=1
° # KERNEL: =====
° # KERNEL: Instruction 11 : ADD R15, R3, R0
° # KERNEL: =====
° # KERNEL: Cycle38 : stage:IF
° # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
° # KERNEL:   Pipeline Registers: PC=11 | IR=00100100001100000000000000000000
° # KERNEL: =====
° # KERNEL: Cycle39 : stage:ID
° # KERNEL:   Control Signals   : AWrite=1 | BWrite=1
° # KERNEL:   Pipeline Registers: IR=00000011110011000000000000000000 | A=11 | B=0
° # KERNEL: =====
° # KERNEL: Cycle40 : stage:EX
° # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=0 | ALUOutWrite=1
° # KERNEL:   Pipeline Registers: A=13 | B=0 | ALUOut=1
° # KERNEL: =====
° # KERNEL: Cycle41 : stage:WB
° # KERNEL:   Control Signals   : RegDst=01 | WBData=00 | RegWr=1
° # KERNEL:   Pipeline Registers: ALUOut=13 | MDR=0 | WriteReg=R15 | BusW=13

```

Figure 16 : JR R12

IF: The processor fetches the JR R12 instruction at PC=9. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path during the fetch cycle. Since pc_update=1, the PC initially advances sequentially. In the printed output, the IR still shows the previous CLR R11 instruction because the new JR instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as JR R12. In the testbench, register R12 was initialized with the value 11, which is used as the jump target address. The control signals AWrite=1 and BWrite=1 allow the register file outputs to be stored into the temporary registers A and B. Since this is a jump-register instruction, the processor reads the value stored inside R12 and stores it into A.

EX: In the execute cycle, the value read from R12 is now available in register A, so A=11. For JR, the ALU result is not needed, so ALUOutWrite=0. Instead, the processor uses the value in A as the next PC value. Therefore, the PC is updated to 11. This explains why the next fetched instruction appears at PC=11 instead of PC=10.

After JR: Since JR R12 updates the PC to 11, the instruction at address 10 is skipped. The processor then fetches the instruction stored at address 11, which is ADD R15, R3, R0.

2.1.1 Final Result:

The results show that the ADD, SUB, AND, OR, XOR, NOR, SLL, and SRL instructions all executed correctly. These R-type instructions follow the same multicycle sequence. In the IF stage, the instruction is fetched from instruction memory. In the ID stage, the source registers are read and stored into the temporary registers A and B. In the EX stage, the ALU performs the selected operation based on ALUOp. Finally, in the WB stage, the result stored in ALUOut is written back to the destination register. Therefore, each of these R-type instructions takes four cycles to complete.

The CLR R11 instruction also executed correctly. It is a special instruction that clears a destination register by writing zero to it. Since it does not need an ALU operation, it skips the EX stage and takes only three cycles: IF, ID, and WB. In this test, R11 was initialized to 55, and after executing CLR R11, it became 0.

The JR R12 instruction correctly redirected the program flow. It reads the value stored inside R12 and loads it into the PC. Since R12 was initialized to 11, executing JR R12 changed the PC to 11. This caused the instruction at address 10 to be skipped, and the processor continued execution from address 11. The JR instruction takes three cycles: IF, ID, and EX, because it updates the PC but does not write any result back to the register file.

Overall, these results confirm that the datapath, control signals, register file, ALU, write-back logic, and PC update mechanism are working correctly in the multicycle processor.

2.2 I-Type Instructions Test

The following table shows the I-Type, memory, and branch instruction sequence used in the test program. Each row contains the instruction address, the 32-bit binary machine code, the decoded assembly instruction, and the operation performed by the processor.

Table 18 : I Type Instructions Test program

Address	Binary Instruction	Instruction	Meaning
16	00101000110001000000000000000101	ADDI R3, R1, 5	$R3 = R1 + 5$
17	00101101000001000000000000000011	ANDI R4, R1, 3	$R4 = R1 \& 3$
18	00110001010001000000000000000011	ORI R5, R1, 3	$R5 = R1 3$
19	00110101101101000000000000000010	LW R6, 2(R13)	$R6 = \text{Mem}[R13 + 2]$
20	00111000101101000000000000000011	SW R2, 3(R13)	$\text{Mem}[R13 + 3] = R2$
21	00111101111101000000000000000100	SWAP R7, 4(R13)	$R7 \leftrightarrow \text{Mem}[R13 + 4]$
22	01000000010001000000000000000001	BEQ R1, R1, 1	If $R1 == R1$, branch and skip address 23
23	00101010100000000000000001101111	ADDI R10, R0, 111	$R10 = 111$ — skipped by BEQ
24	01000110010001000000000000000001	BNE R9, R1, 1	If $R9 \neq R1$, branch and skip address 25
25	001010101100000000000000011011110	ADDI R11, R0, 222	$R11 = 222$ — skipped by BNE
26	00101011000000000000000001111011	ADDI R12, R0, 123	$R12 = 123$ marker
27	00101011100000000000000000000000	ADDI R14, R0, 0	$R14 = 0$ filler

Before running the I-Type instruction sequence, the testbench initializes the needed registers and memory locations. In this test, $R1=10$, $R2=3$, $R12=11$, and $R13=20$. Register $R13$ is used as the base register for memory instructions. The data memory is also initialized so that $\text{Mem}[22]=99$, $\text{Mem}[23]=0$, and $\text{Mem}[24]=88$. These values are used to verify the LW, SW, and SWAP instructions.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 16 : ADDI R3, R1, 5
◦ # KERNEL: =====
◦ # KERNEL: Cycle54 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=16 | IR=00100000000000000000000000000000
◦ # KERNEL:
◦ # KERNEL: Cycle55 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=0
◦ # KERNEL:   Pipeline Registers: IR=001010001100010000000000000000101 | A=0 | B=0 | Imm32=5
◦ # KERNEL:
◦ # KERNEL: Cycle56 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=1 | ALUOp=0 | ALUOutWrite=1 | PCSrc=00
◦ # KERNEL:   Pipeline Registers: A=10 | B=0 | Imm32=5 | ALUOut=13
◦ # KERNEL:
◦ # KERNEL: Cycle57 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=00 | WBData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=15 | MDR=0 | WriteReg=R3 | BusW=15
◦ # KERNEL:

```

Figure 17: ADDI R3, R1, 5

IF: The processor fetches the instruction at PC=16. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as ADDI R3, R1, 5. The control signals AWrite=1 and BWrite=1 allow the register file outputs to be stored into the temporary registers A and B. In this instruction, R1 is the source register, so A receives the value of R1 and the immediate is zero-extended to 32 bits. Therefore, the immediate value becomes Imm32=5.

EX: The value read in the previous ID cycle is now available in register A. Since ALUSrc=1, the ALU uses the immediate value Imm32 as the second input instead of register B. The control signal ALUOp=0 selects the ADD operation. Therefore, the ALU computes $10 + 5 = 15$. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle. The printed ALUOut value is the old value before the new ADDI result is latched.

WB: The ALU result is now stored in ALUOut, and its value is 15. The control signal RegWr=1 enables writing to the register file. Since RegDst=00, the destination register is selected as R3, and since WBData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 15 into register R3. This confirms that ADDI R3, R1, 5 executed correctly because $R3 = R1 + 5 = 10 + 5 = 15$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 17 : ANDI R4, R1, 3
◦ # KERNEL: =====
◦ # KERNEL: Cycle58 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=17 | IR=00101000110001000000000000000101
◦ # KERNEL:
◦ # KERNEL: Cycle59 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=0
◦ # KERNEL:   Pipeline Registers: IR=00101101000001000000000000000011 | A=10 | B=0 | Imm32=3
◦ # KERNEL:
◦ # KERNEL: Cycle60 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=1 | ALUOp=2 | ALUOutWrite=1 | PCSrc=00
◦ # KERNEL:   Pipeline Registers: A=10 | B=0 | Imm32=3 | ALUOut=15
◦ # KERNEL:
◦ # KERNEL: Cycle61 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=00 | WBData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=2 | MDR=0 | WriteReg=R4 | BusW=2
◦ # KERNEL:

```

Figure 18: ANDI R4, R1, 3

IF: The processor fetches the instruction at PC=17. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous ADDI instruction because the new instruction is latched at the end of the IF cycle.

ID: The instruction stored in IR is decoded as ANDI R4, R1, 3. The control signals AWrite=1 and BWrite=1 allow the source register value to be stored into A. In this instruction, R1 is the source register, so A receives the value 10 and the immediate is zero-extended to 32 bits. Therefore, the immediate value becomes Imm32=3.

EX: The value of R1 is now available in A, and the immediate value is available in Imm32. Since ALUSrc=1, the ALU uses Imm32 instead of register B. The control signal ALUOp=2 selects the AND operation. Therefore, the ALU computes 10 AND 3. In binary, 10 = 1010 and 3 = 0011, so the result is 0010, which equals 2. The signal ALUOutWrite=1 allows this result to be stored in ALUOut at the end of the cycle.

WB: The ALU result is now stored in ALUOut, and its value is 2. The control signal RegWr=1 enables writing to the register file. Since RegDst=00, the destination register is selected as R4, and since WBData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 2 into register R4. This confirms that ANDI R4, R1, 3 executed correctly because $R4 = R1 \text{ AND } 3 = 10 \text{ AND } 3 = 2$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 18 : ORI R5, R1, 3
◦ # KERNEL: =====
◦ # KERNEL: Cycle62 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=18 | IR=00101101000000100000000000000011
◦ # KERNEL:
◦ # KERNEL: Cycle63 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=0
◦ # KERNEL:   Pipeline Registers: IR=00110001010001000000000000000011 | A=10 | B=0 | Imm32=3
◦ # KERNEL:
◦ # KERNEL: Cycle64 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=1 | ALUOp=3 | ALUOutWrite=1 | PCSrc=00
◦ # KERNEL:   Pipeline Registers: A=10 | B=0 | Imm32=3 | ALUOut=2
◦ # KERNEL:
◦ # KERNEL: Cycle65 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=00 | WBData=00 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=11 | MDR=0 | WriteReg=R5 | BusW=11
◦ # KERNEL:

```

Figure 19: ORI R5, R1, 3

IF: The processor fetches the instruction at PC=18. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially.

ID: The instruction stored in IR is decoded as ORI R5, R1, 3. The control signals AWrite=1 and BWrite=1 allow the register file outputs to be stored into A and B. R1 is the source register, so A receives the value 10 and B the immediate is zero-extended to 32 bits. Therefore, the immediate value becomes Imm32=3.

EX: The value of R1 is now available in A, and Imm32 contains the immediate value 3. Since ALUSrc=1, the ALU uses Imm32 as the second input. The control signal ALUOp=3 selects the OR operation. Therefore, the ALU computes 10 OR 3. In binary, 10 = 1010 and 3 = 0011, so the result is 1011, which equals 11. ALUOutWrite=1 stores this result in ALUOut.

WB: The ALU result is now stored in ALUOut, and its value is 11. The control signal RegWr=1 enables writing to the register file. Since RegDst=00, the destination register is selected as R5, and since WBData=00, the data written back comes from ALUOut. Therefore, the processor writes the value 11 into register R5. This confirms that ORI R5, R1, 3 executed correctly because $R5 = R1 \text{ OR } 3 = 10 \text{ OR } 3 = 11$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 19 : LW R6, 2(R13)
◦ # KERNEL: =====
◦ # KERNEL: Cycle66 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=19 | IR=00110001010001000000000000000011
◦ # KERNEL:
◦ # KERNEL: Cycle67 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=0
◦ # KERNEL:   Pipeline Registers: IR=00110101101101000000000000000010 | A=10 | B=0 | Imm32=2
◦ # KERNEL:
◦ # KERNEL: Cycle68 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=1 | ALUOp=0 | ALUOutWrite=1 | PCSrc=00
◦ # KERNEL:   Pipeline Registers: A=20 | B=0 | Imm32=2 | ALUOut=11
◦ # KERNEL:
◦ # KERNEL: Cycle69 : stage:MEM
◦ # KERNEL:   Control Signals   : MemRd=1 | MemWr=0 | MDRWrite=1
◦ # KERNEL:   Pipeline Registers: ALUOut=22 | B=0 | MDR=0 | MemDataOut=99
◦ # KERNEL:
◦ # KERNEL: Cycle70 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=00 | WBData=01 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=22 | MDR=99 | WriteReg=R6 | BusW=99
◦ # KERNEL:

```

Figure 20: LW R6, 2(R13)

IF: The processor fetches the instruction at PC=19. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous instruction because the new LW instruction is latched into IR at the end of the IF cycle.

ID: The instruction stored in IR is decoded as LW R6, 2(R13). The control signals AWrite=1 and BWrite=1 allow the register file outputs to be stored into the temporary registers A and B. In this instruction, R13 is the base register, so A receives the value of R13, which is 20. The immediate field is selected because ExtSel=0, and since ExtOp=0, the immediate is zero-extended to 32 bits. Therefore, the immediate value becomes Imm32=2. In the printed output, A=10 and B=0 still appear because these are old values before the new register values are latched at the end of the ID cycle.

EX: The base address read in the previous ID cycle is now available in register A, so A=20. The offset is available in Imm32, and its value is 2. Since ALUSrc=1, the ALU uses Imm32 as the second input instead of register B. The control signal ALUOp=0 selects the ADD operation. Therefore, the ALU computes the effective memory address $20 + 2 = 22$. ALUOutWrite=1 allows this address to be stored in ALUOut at the end of the cycle. In the printed output, ALUOut=11 is the old value from the previous ORI instruction before the new address is latched.

MEM: The calculated address is now stored in ALUOut, and its value is 22. The control signal MemRd=1 enables reading from data memory, while MemWr=0 means no memory write occurs. Since MDRWrite=1, the data read from memory is stored into MDR at the end of the MEM cycle. The memory location Mem[22] contains 99, so MemDataOut=99, and MDR receives 99. In the printed output, MDR=0 is the old value before the new memory data is latched.

WB: The loaded value is now stored in MDR, and its value is 99. The control signal RegWr=1 enables writing to the register file. Since RegDst=00, the destination register is selected as R6, and since WBData=01, the write-back data comes from MDR instead of ALUOut. Therefore,

BusW=99, and the processor writes the value 99 into register R6. This confirms that LW R6, 2(R13) executed correctly because $R6 = \text{Mem}[R13 + 2] = \text{Mem}[20 + 2] = \text{Mem}[22] = 99$.

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 20 : SW R2, 3(R13)
◦ # KERNEL: =====
◦ # KERNEL: Cycle71 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=20 | IR=0011010110110100000000000000010
◦ # KERNEL:
◦ # KERNEL: Cycle72 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=0
◦ # KERNEL:   Pipeline Registers: IR=00111000101101000000000000000011 | A=20 | B=0 | Imm32=3
◦ # KERNEL:
◦ # KERNEL: Cycle73 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=1 | ALUOp=0 | ALUOutWrite=1 | PCSrc=00
◦ # KERNEL:   Pipeline Registers: A=20 | B=3 | Imm32=3 | ALUOut=22
◦ # KERNEL:
◦ # KERNEL: Cycle74 : stage:MEM
◦ # KERNEL:   Control Signals   : MemRd=0 | MemWr=1 | MDRWrite=0
◦ # KERNEL:   Pipeline Registers: ALUOut=23 | B=3 | MDR=99 | MemDataOut=0
◦ # KERNEL:

```

Figure 21: SW R2, 3(R13)

IF: The processor fetches the instruction at PC=20. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous LW instruction because the new SW instruction is latched into IR at the end of the IF cycle.

ID: The instruction stored in IR is decoded as SW R2, 3(R13). The control signals AWrite=1 and BWrite=1 allow the register file outputs to be stored into the temporary registers A and B. In this instruction, R13 is the base register, so A receives the value of R13, which is 20. Register R2 contains the value that will be stored in memory, so B receives the value of R2, which is 3. The immediate field is selected because ExtSel=0, and since ExtOp=0, the immediate is zero-extended to 32 bits. Therefore, the immediate value becomes Imm32=3. In the printed output, A=20 and B=0 still appear because B has not been latched with the new value yet at the time of printing.

EX: The base address read in the previous ID cycle is now available in register A, so A=20. The value to be stored is available in register B, so B=3. The offset is available in Imm32, and its value is 3. Since ALUSrc=1, the ALU uses Imm32 as the second input instead of register B. The control signal ALUOp=0 selects the ADD operation. Therefore, the ALU computes the effective memory address $20 + 3 = 23$. ALUOutWrite=1 allows this address to be stored in ALUOut at the end of the cycle. In the printed output, ALUOut=22 is the old value from the previous LW instruction before the new address is latched.

MEM: The calculated address is now stored in ALUOut, and its value is 23. The control signal MemWr=1 enables writing to data memory, while MemRd=0 means no memory read occurs. Since MDRWrite=0, the MDR register is not updated during this instruction. The value stored in B is 3,

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 21 : SWAP R7, 4(R13)
◦ # KERNEL: =====
◦ # KERNEL: Cycle75 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=21 | IR=00111000101101000000000000000011
◦ # KERNEL:
◦ # KERNEL: Cycle76 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=0
◦ # KERNEL:   Pipeline Registers: IR=0011101111101000000000000000100 | A=20 | B=3 | Imm32=4
◦ # KERNEL:
◦ # KERNEL: Cycle77 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=1 | ALUOp=0 | ALUOutWrite=1 | PCSrc=00
◦ # KERNEL:   Pipeline Registers: A=20 | B=9 | Imm32=4 | ALUOut=23
◦ # KERNEL:
◦ # KERNEL: Cycle78 : stage:MEM
◦ # KERNEL:   Control Signals   : MemRd=1 | MemWr=1 | MDRWrite=1
◦ # KERNEL:   Pipeline Registers: ALUOut=24 | B=9 | MDR=99 | MemDataOut=88
◦ # KERNEL:
◦ # KERNEL: Cycle79 : stage:WB
◦ # KERNEL:   Control Signals   : RegDst=00 | WBDData=01 | RegWr=1
◦ # KERNEL:   Pipeline Registers: ALUOut=24 | MDR=88 | WriteReg=R7 | BusW=88
◦ # KERNEL:

```

Figure 22: SWAP R7, 4(R13)

so the processor writes 3 into Mem[23]. This confirms that SW R2, 3(R13) executed correctly because $\text{Mem}[\text{R13} + 3] = \text{Mem}[20 + 3] = \text{Mem}[23] = \text{R2} = 3$.

IF: The processor fetches the instruction at PC=21. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous SW instruction because the new SWAP instruction is latched into IR at the end of the IF cycle.

ID: The instruction stored in IR is decoded as SWAP R7, 4(R13). The control signals AWrite=1 and BWrite=1 allow the register file outputs to be stored into the temporary registers A and B. In this instruction, R13 is the base register, so A receives the value of R13, which is 20. Register R7 contains the value that will be swapped with memory, so B receives the value of R7. At this point, R7 contains 9 because the earlier XOR R7, R1, R2 instruction changed R7 to $10 \text{ XOR } 3 = 9$. The immediate field is selected because ExtSel=0, and since ExtOp=0, the immediate is zero-extended to 32 bits. Therefore, the immediate value becomes Imm32=4. In the printed output, A=20 and B=3 still appear because these are old values before the new B value is latched at the end of the ID cycle.

EX: The base address read in the previous ID cycle is now available in register A, so A=20. The old value of R7 is now available in register B, so B=9. The offset is available in Imm32, and its value is 4. Since ALUSrc=1, the ALU uses Imm32 as the second input instead of register B. The control signal ALUOp=0 selects the ADD operation. Therefore, the ALU computes the effective memory address $20 + 4 = 24$. ALUOutWrite=1 allows this address to be stored in ALUOut at the end of the cycle. In the printed output, ALUOut=23 is the old value from the previous SW instruction before the new address is latched.

MEM: The calculated address is now stored in ALUOut, and its value is 24. For the SWAP instruction, both MemRd=1 and MemWr=1 are asserted. MemRd=1 reads the old value from data memory, and MemWr=1 writes the old register value into memory. Since MDRWrite=1, the memory output is stored into MDR at the end of the MEM cycle. The memory location Mem[24] contains 88, so MemDataOut=88. At the same time, the value stored in B is 9, so the processor writes 9 into Mem[24]. In the printed output, MDR=99 is the old value from the previous LW instruction before the new memory value 88 is latched.

WB: The old memory value is now stored in MDR, and its value is 88. The control signal RegWr=1 enables writing to the register file. Since RegDst=00, the destination register is selected as R7, and since WBDData=01, the write-back data comes from MDR. Therefore, BusW=88, and the processor writes 88 into register R7. This confirms that SWAP R7, 4(R13) executed correctly because R7 receives the old memory value 88, while Mem[R13 + 4] = Mem[24] receives the old R7 value 9

```

◦ # KERNEL: =====
◦ # KERNEL: Instruction 22 : BEQ R1, R1, 1
◦ # KERNEL: =====
◦ # KERNEL: Cycle80 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=22 | IR=001111011110100000000000000100
◦ # KERNEL:
◦ # KERNEL: Cycle81 : stage:ID
◦ # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=0
◦ # KERNEL:   Pipeline Registers: IR=01000000010001000000000000000001 | A=20 | B=9 | Imm32=1
◦ # KERNEL:
◦ # KERNEL: Cycle82 : stage:EX
◦ # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=1 | ALUOutWrite=0 | PCSrc=01
◦ # KERNEL:   Pipeline Registers: A=10 | B=10 | Imm32=1 | ALUOut=24
◦ # KERNEL:
◦ # KERNEL: =====
◦ # KERNEL: Instruction 23 : BNE R9, R1, 1
◦ # KERNEL: =====
◦ # KERNEL: Cycle83 : stage:IF
◦ # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
◦ # KERNEL:   Pipeline Registers: PC=24 | IR=01000000010001000000000000000001
◦ # KERNEL:

```

Figure 23: BEQ R1, R1, 1

IF: The processor fetches the instruction at PC=22. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous SWAP instruction because the new BEQ instruction is latched into IR at the end of the IF cycle.

ID: The instruction stored in IR is decoded as BEQ R1, R1, 1. The control signals AWrite=1 and BWrite=1 allow the register file outputs to be stored into the temporary registers A and B. In this instruction, both registers being compared are R1, so A receives the value of R1 and B also receives the value of R1. Since R1 contains 10, both A and B should become 10. The immediate field is selected because ExtSel=0, and since ExtOp=0, the immediate is zero-extended to 32 bits.

Therefore, the immediate value becomes Imm32=1. In the printed output, A=20 and B=9 still appear because these are old values from the previous SWAP instruction before the new values are latched at the end of the ID cycle.

EX: The compared register values read in the previous ID cycle are now available in the pipeline registers, so A=10 and B=10. Since ALUSrc=0, the ALU uses register B as the second input instead of the immediate value. The control signal ALUOp=1 selects the SUB operation. Therefore, the ALU compares the two registers by computing $10 - 10 = 0$. Since the result is zero, the Zero flag becomes 1. For the BEQ instruction, when Zero=1, the branch is taken and PCSrc=01 selects the branch target path.

The branch immediate is 1. At this point, the PC has already advanced past the BEQ instruction, so the branch target becomes PC + Imm32. This causes the processor to skip the instruction at address 23 and continue execution at address 24. This is shown in the log because the next fetched instruction is BNE R9, R1, 1 at PC=24. Therefore, BEQ R1, R1, 1 executed correctly because R1 equals R1, the branch was taken, and address 23 was skipped.

```

.. -----
o # KERNEL: =====
o # KERNEL: Instruction 23 : BNE R9, R1, 1
o # KERNEL: =====
o # KERNEL: Cycle83 : stage:IF
o # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
o # KERNEL:   Pipeline Registers: PC=24 | IR=01000000010001000000000000000001
o # KERNEL:
o # KERNEL: Cycle84 : stage:ID
o # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=0
o # KERNEL:   Pipeline Registers: IR=01000110010001000000000000000001 | A=10 | B=10 | Imm32=1
o # KERNEL:
o # KERNEL: Cycle85 : stage:EX
o # KERNEL:   Control Signals   : ALUSrc=0 | ALUOp=1 | ALUOutWrite=0 | PCSrc=01
o # KERNEL:   Pipeline Registers: A=10 | B=24 | Imm32=1 | ALUOut=24
o # KERNEL:
o # KERNEL: =====
o # KERNEL: Instruction 24 : ADDI R12, R0, 123
o # KERNEL: =====
o # KERNEL: Cycle86 : stage:IF
o # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
o # KERNEL:   Pipeline Registers: PC=26 | IR=01000110010001000000000000000001
.. -----

```

Figure 24: BNE R9, R1, 1

IF: The processor fetches the instruction at PC=24. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path. Since pc_update=1, the PC advances sequentially. In the printed output, the IR still shows the previous BEQ instruction because the new BNE instruction is latched into IR at the end of the IF cycle.

ID: The instruction stored in IR is decoded as BNE R9, R1, 1. The control signals AWrite=1 and BWrite=1 allow the register file outputs to be stored into the temporary registers A and B. In this instruction, R1 is read into A, and R9 is read into B. R1 contains 10, while R9 contains 24 from the earlier SLL R9, R2, R2 instruction. The immediate field is selected because ExtSel=0, and

since ExtOp=0, the immediate is zero-extended to 32 bits. Therefore, the immediate value becomes Imm32=1. In the printed output, A=10 and B=10 still appear because these are old values from the previous BEQ instruction before the new B value is latched at the end of the ID cycle.

EX: The compared register values read in the previous ID cycle are now available in the pipeline registers, so A=10 and B=24. Since ALUSrc=0, the ALU uses register B as the second input instead of the immediate value. The control signal ALUOp=1 selects the SUB operation. Therefore, the ALU compares the two registers by computing $10 - 24$, which is not equal to zero. Since the result is not zero, the Zero flag becomes 0. For the BNE instruction, when Zero=0, the branch is taken and PCSrc=01 selects the branch target path.

The branch immediate is 1. At this point, the PC has already advanced past the BNE instruction, so the branch target becomes $PC + Imm32$. This causes the processor to skip the instruction at address 25 and continue execution at address 26. This is shown in the log because the next fetched instruction is ADDI R12, R0, 123 at PC=26. Therefore, BNE R9, R1, 1 executed correctly because R9 is not equal to R1, the branch was taken, and address 25 was skipped.

2.2.1 Final Result:

The results show that the ADDI, ANDI, and ORI instructions executed correctly. These I-type ALU instructions follow the same multicycle sequence as the R-type ALU instructions: IF, ID, EX, and WB. In the IF stage, the instruction is fetched from instruction memory. In the ID stage, the source register is read and the immediate value is extended. In the EX stage, ALUSrc=1 selects the immediate value as the second ALU input, and the ALU performs the required operation. In the WB stage, the result stored in ALUOut is written back to the destination register.

The LW instruction also executed correctly. It uses five cycles: IF, ID, EX, MEM, and WB. In the EX stage, the ALU computes the memory address using the base register and immediate offset. In the MEM stage, the data memory is read, and the value is stored into MDR. In the WB stage, the value from MDR is written into the destination register. In this test, LW R6, 2(R13) loaded the value from Mem[22], which was 99, into R6.

The SW instruction executed correctly. It uses four cycles: IF, ID, EX, and MEM. In the EX stage, the ALU computes the target memory address. In the MEM stage, MemWr=1 enables writing the register value into data memory. In this test, SW R2, 3(R13) stored the value 3 into Mem[23].

The SWAP instruction also executed correctly. It uses five cycles: IF, ID, EX, MEM, and WB. In the MEM stage, both MemRd and MemWr are asserted, so the processor reads the old memory value and writes the old register value to memory. In the WB stage, the old memory value stored in MDR is written back to the register file. In this test, SWAP R7, 4(R13) wrote 88 into R7 and wrote 9 into Mem[24]. The value written to Mem[24] is 9 because R7 had already been changed by the earlier XOR instruction.

The BEQ and BNE instructions also executed correctly. Both branch instructions use three cycles: IF, ID, and EX. In the EX stage, the ALU performs subtraction to compare the two register values. For BEQ R1, R1, 1, the subtraction result is zero, so the branch is taken and the instruction at address 23 is skipped. For BNE R9, R1, 1, the subtraction result is not zero, so the branch is taken and the instruction at address 25 is skipped. The processor then reaches address 26 and executes ADDI R12, R0, 123, which writes 123 into R12. This confirms that the branch mechanism and PC update logic are working correctly.

Overall, these results confirm that the I-type ALU datapath, immediate extension, memory access logic, SWAP operation, branch comparison logic, write-back mux, and PC update mechanism are working correctly in the multicycle processor.

2.3 J-Type Instructions Test

Table 19: J Type Instructions Test program

Address	Binary Instruction	Instruction	Meaning
32	0100100000000000000000000100010	J 34	PC = 34, skip address 33
33	00101010100000000000000001101111	ADDI R10, R0, 111	R10 = 111 — skipped by J
34	0100110000000000000000000100100	JAL 36	PC = 36, R14 = 35
35	00101010110000000000000001101110	ADDI R11, R0, 222	R11 = 222 — skipped by JAL
36	0010101101000000000000000110111	ADDI R13, R0, 55	R13 = 55 marker

```

° # KERNEL: =====
° # KERNEL: Instruction 30 : J 34
° # KERNEL: =====
° # KERNEL: Cycle110 : stage:IF
° # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
° # KERNEL:   Pipeline Registers: PC=32 | IR=00101011100000000000000000000000
° # KERNEL: |
° # KERNEL: Cycle111 : stage:ID
° # KERNEL:   Control Signals   : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=1
° # KERNEL:   Pipeline Registers: IR=0100100000000000000000000000100010 | A=0 | B=0 | Imm32=34
° # KERNEL: =====
° # KERNEL: Instruction 31 : JAL 36
° # KERNEL: =====
° # KERNEL: Cycle112 : stage:IF
° # KERNEL:   Control Signals   : IRWrite=1 | PCSrc=00 | pc_update=1
° # KERNEL:   Pipeline Registers: PC=34 | IR=0100100000000000000000000000100010
° # KERNEL: =====

```

Figure 25: J 34

IF: The processor fetches the instruction at PC=32. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path during the fetch cycle. Since pc_update=1, the PC initially advances sequentially. In the printed output, the IR still shows the previous filler instruction because the new J instruction is latched into IR at the end of the IF cycle.

ID: The instruction stored in IR is decoded as J 34. This is a jump-type instruction, so it does not need to read source register values for an ALU operation. The control signals AWrite=1 and BWrite=1 may still be asserted, but their values are not important for the jump operation. Since this instruction uses the 26-bit jump offset, ExtSel=1 selects the offset field instead of the 18-bit immediate field. The value is extended to Imm32=34, which represents the jump target address.

EX: The J instruction updates the program counter using the jump target. For this instruction, PCSrc selects the jump path, so the next PC becomes Imm32. Therefore, the processor changes the PC to 34. This causes the instruction at address 33 to be skipped.

After J: The next fetched instruction is JAL 36 at PC=34. This confirms that J 34 executed correctly because the processor jumped from address 32 to address 34 and skipped the instruction at address 33.

```

0 # KERNEL: =====
0 # KERNEL: Instruction 31 : JAL 36
0 # KERNEL: =====
0 # KERNEL: Cycle112 : stage:IF
0 # KERNEL: Control Signals : IRWrite=1 | PCSrc=00 | pc_update=1
0 # KERNEL: Pipeline Registers: PC=34 | IR=01001000000000000000000000000000100010
0 # KERNEL:
0 # KERNEL: Cycle113 : stage:ID
0 # KERNEL: Control Signals : AWrite=1 | BWrite=1 | ExtOp=0 | ExtSel=1
0 # KERNEL: Pipeline Registers: IR=01001100000000000000000000000000100100 | A=0 | B=0 | Imm32=36
0 # KERNEL:
0 # KERNEL: Cycle114 : stage:WB
0 # KERNEL: Control Signals : RegDst=10 | WBData=10 | RegWr=1
0 # KERNEL: Pipeline Registers: ALUOut=0 | MDR=88 | WriteReg=R14 | BusW=35
0 # KERNEL:
0 # KERNEL: =====
0 # KERNEL: Instruction 32 : ADDI R13, R0, 55
0 # KERNEL: =====
0 # KERNEL: Cycle115 : stage:IF
0 # KERNEL: Control Signals : IRWrite=1 | PCSrc=00 | pc_update=1
0 # KERNEL: Pipeline Registers: PC=36 | IR=01001100000000000000000000000000100100

```

Figure 26: JAL 36

IF: The processor fetches the instruction at PC=34. IRWrite=1 allows the instruction to be loaded into IR, while PCSrc=00 selects the normal PC+1 path during the fetch cycle. Since pc_update=1, the PC initially advances sequentially. In the printed output, the IR already shows the JAL instruction encoding, which corresponds to JAL 36.

ID: The instruction stored in IR is decoded as JAL 36. This is a jump-and-link instruction, so it uses the 26-bit jump offset instead of the 18-bit immediate field. Since ExtSel=1, the processor selects the jump offset field, and the extended value becomes Imm32=36. The values in A and B are not important for this instruction because JAL does not perform an ALU operation using source registers.

WB: The JAL instruction writes the return address into register R14. The control signal RegWr=1 enables writing to the register file. Since RegDst=10, the destination register is selected as R14. Since WBData=10, the write-back data comes from PCsave, which holds the return address. In this test, the return address is 35, so BusW=35 and the processor writes 35 into R14.

At the same time, the processor updates the PC to the jump target address 36. This causes the instruction at address 35 to be skipped. The next fetched instruction is `ADDI R13, R0, 55` at PC=36, which confirms that `JAL 36` executed correctly. Therefore, `JAL` both saved the return address `R14=35` and jumped to address 36.

2.3.1 Final Result:

The results show that the J and JAL instructions executed correctly. The J instruction is an unconditional jump instruction. In this test, the instruction J 34 was fetched at address 32. The processor selected the 26-bit jump offset because ExtSel=1, and the extended value became Imm32=34. The PC was then updated to 34, so the instruction at address 33 was skipped. The next fetched instruction was JAL 36 at PC=34, which confirms that the J instruction redirected the program flow correctly.

The JAL instruction also executed correctly. In this test, the instruction JAL 36 was fetched at address 34. The processor selected the 26-bit jump offset, so Imm32 became 36. The instruction then jumped to address 36 and skipped the instruction at address 35. At the same time, JAL saved the return address into register R14. Since the next sequential address after PC=34 is 35, the value written to R14 was 35. This is shown by the write-back stage, where WriteReg=R14 and BusW=35.

After the jump, the processor fetched ADDI R13, R0, 55 at PC=36. This instruction worked as a marker to prove that the JAL target was reached. It wrote the value 55 into R13. Therefore, the final results are R14=35 and R13=55. R14=35 proves that the link operation worked correctly, and R13=55 proves that the processor jumped to the correct target address.

Overall, these results confirm that the jump control logic, the 26-bit offset extension, the PC update mechanism, the JAL return-address write-back path, and the destination selection for R14 are working correctly in the multicycle processor.

3. Waveform Analysis

3.1 R-Type Instructions Waveform

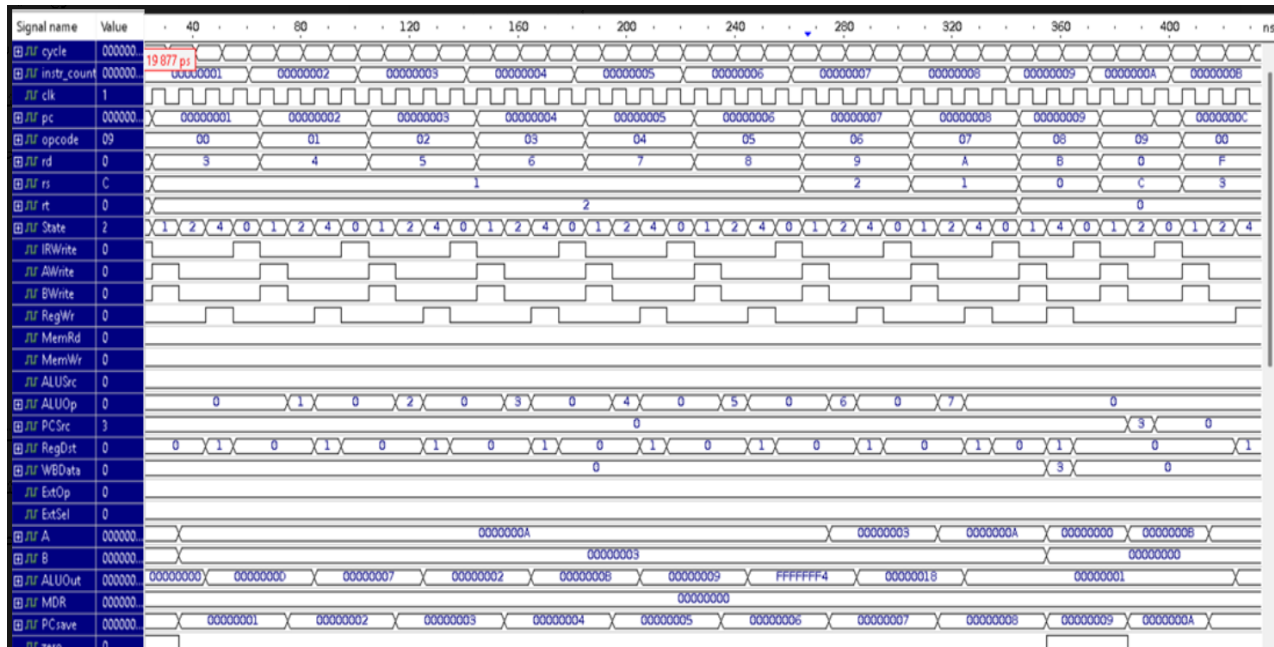


Figure 27: R-Type Instructions Waveform

The first waveform shows the execution of the R-Type instructions. The PC increases sequentially from one instruction address to the next, and the opcode changes according to each R-Type instruction such as ADD, SUB, AND, OR, XOR, NOR, SLL, JR, CLR and SRL. For these instructions, the processor goes through the normal multicyle stages: IF, ID, EX, and WB. This is shown by the State signal changing between 0, 1, 2, and 4.

During the IF stage, IRWrite is asserted so the instruction is loaded into the instruction register. During the ID stage, AWrite and BWrite are asserted, so the values of the source registers are stored into A and B. During the EX stage, ALUSrc=0 because the ALU uses register B as the second input, not an immediate value. The ALUOp signal changes depending on the instruction, selecting ADD, SUB, AND, OR, XOR, NOR, SLL, or SRL. Finally, during the WB stage, RegWr=1 and WBDat=00, so the ALUOut result is written back into the destination register. This confirms that the R-Type datapath, ALU control, register read, and write-back logic are working correctly.

3.2 I-Type Instructions Waveform

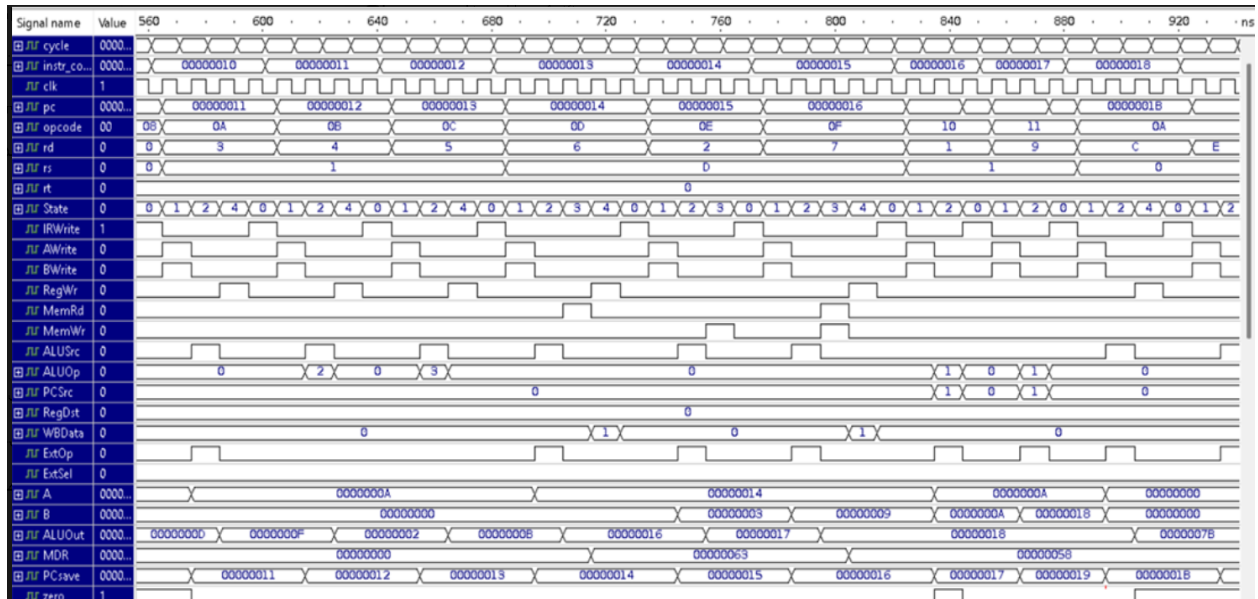


Figure 28 : I-Type, Memory, and Branch Instructions Waveform

The second waveform shows the execution of the I-Type, memory, and branch instructions. The instructions include ADDI, ANDI, ORI, LW, SW, SWAP, BEQ, and BNE. For the I-Type ALU instructions, the State signal shows the sequence IF, ID, EX, and WB. In these instructions, ALUSrc=1, which means the ALU uses the extended immediate value instead of register B. The ExtSel and ExtOp signals select and extend the immediate field, and the Imm32 signal shows the extended immediate value.

For the LW instruction, the ALU first calculates the memory address using the base register and the immediate offset. Then the processor enters the MEM stage, where MemRd=1 and MDRWrite=1, so the value read from data memory is stored into MDR. In the WB stage, WBDData=01 selects MDR as the write-back data, so the loaded value is written into the destination register.

For the SW instruction, MemWr=1 in the MEM stage, so the value stored in register B is written into data memory. For the SWAP instruction, both MemRd and MemWr are asserted, meaning the processor reads the old memory value and writes the old register value into memory. Then WBDData=01 writes the old memory value back into the register.

The branch instructions are also shown in the waveform. BEQ and BNE use ALUSrc=0 and ALUOp=1 to subtract the two compared registers. When the branch condition is true, PCSrc changes to 01, so the PC is updated to the branch target address. This confirms that the immediate

datapath, memory access logic, SWAP operation, branch comparison, and PC branch update are working correctly

3.3 J-Type Instructions Waveform

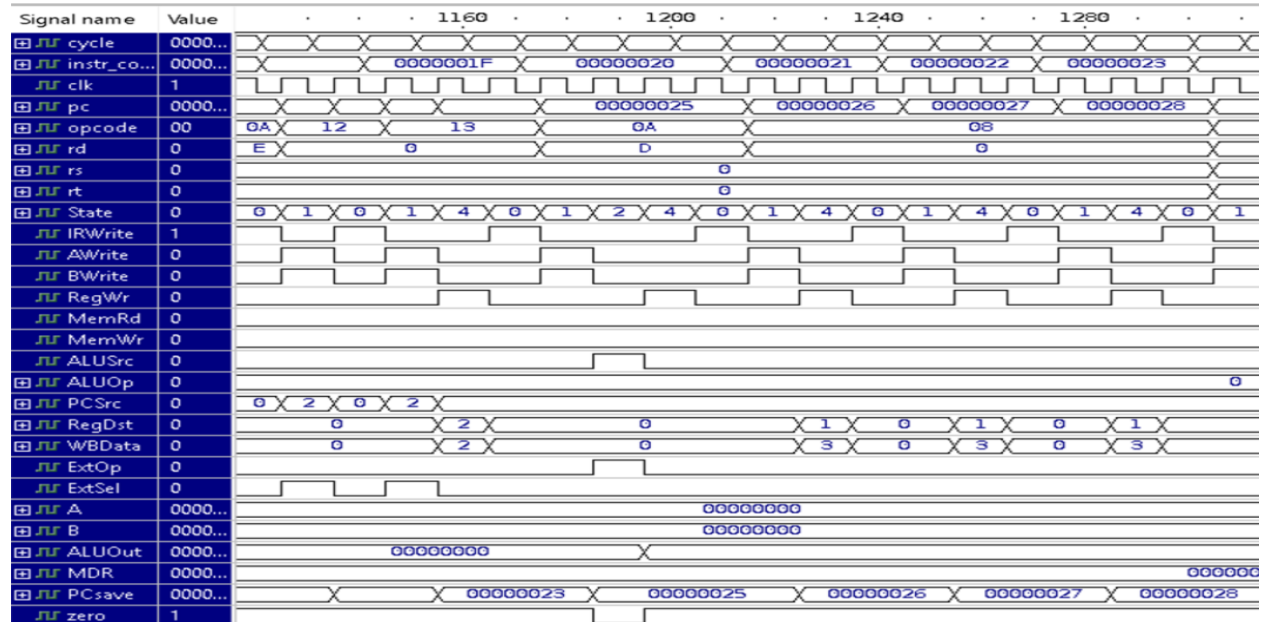


Figure 29 : J-Type Instructions Waveform

The third waveform shows the execution of the jump instructions J and JAL. In this part, the PC reaches the jump instruction and then changes to the target address instead of continuing sequentially. For the J instruction, ExtSel=1, which means the processor selects the 26-bit jump offset instead of the 18-bit immediate field. The extended offset is placed in Imm32, and PCSrc selects the jump path. Therefore, the PC is updated to the jump target address, and the instruction after J is skipped.

The waveform also shows the JAL instruction. JAL works like a jump, but it also saves the return address. During JAL, RegWr=1, RegDst=10, and WBDData=10. This means the processor writes the saved return address into register R14. In the waveform, PCsave holds the return address, and reg_write_data carries this value to the register file. After that, the PC jumps to the target address, where the marker instruction is executed.

This confirms that the J instruction correctly updates the PC to the jump target, and the JAL instruction correctly performs both operations: jumping to the target address and saving the return address in R14. Therefore, the jump control logic, offset extension, PC update path, and JAL write-back path are working correctly.

Challenges

During the project, the team faced several challenges. One of the main challenges was time management, since the project required many stages including datapath design, control unit design, Verilog implementation, testbench development, simulation, debugging, waveform analysis, and report writing. It was sometimes difficult to find enough common time for all team members to work together, especially when different parts of the project depended on each other.

Another challenge was that the practical implementation was more complex than the theoretical design. Although the instruction formats, datapath, and control signals were clear in theory, connecting all components together in Verilog required careful checking and debugging. Some signals behaved differently than expected during simulation, especially because values are updated at the end of the clock cycle in a multicycle processor. This required us to analyze the waveform carefully, trace each instruction through the FSM states, and fix errors step by step.

The testing stage was also challenging because every instruction had to be verified separately and then checked again as part of the full program. Branch, jump, memory, and write-back instructions required special attention because they affect the PC, memory, and register file in different ways. Despite these challenges, the team was able to solve the problems through cooperation, repeated simulation, waveform analysis, and continuous debugging. These challenges helped us gain a deeper understanding of how a multicycle processor works in practice.

Teamwork

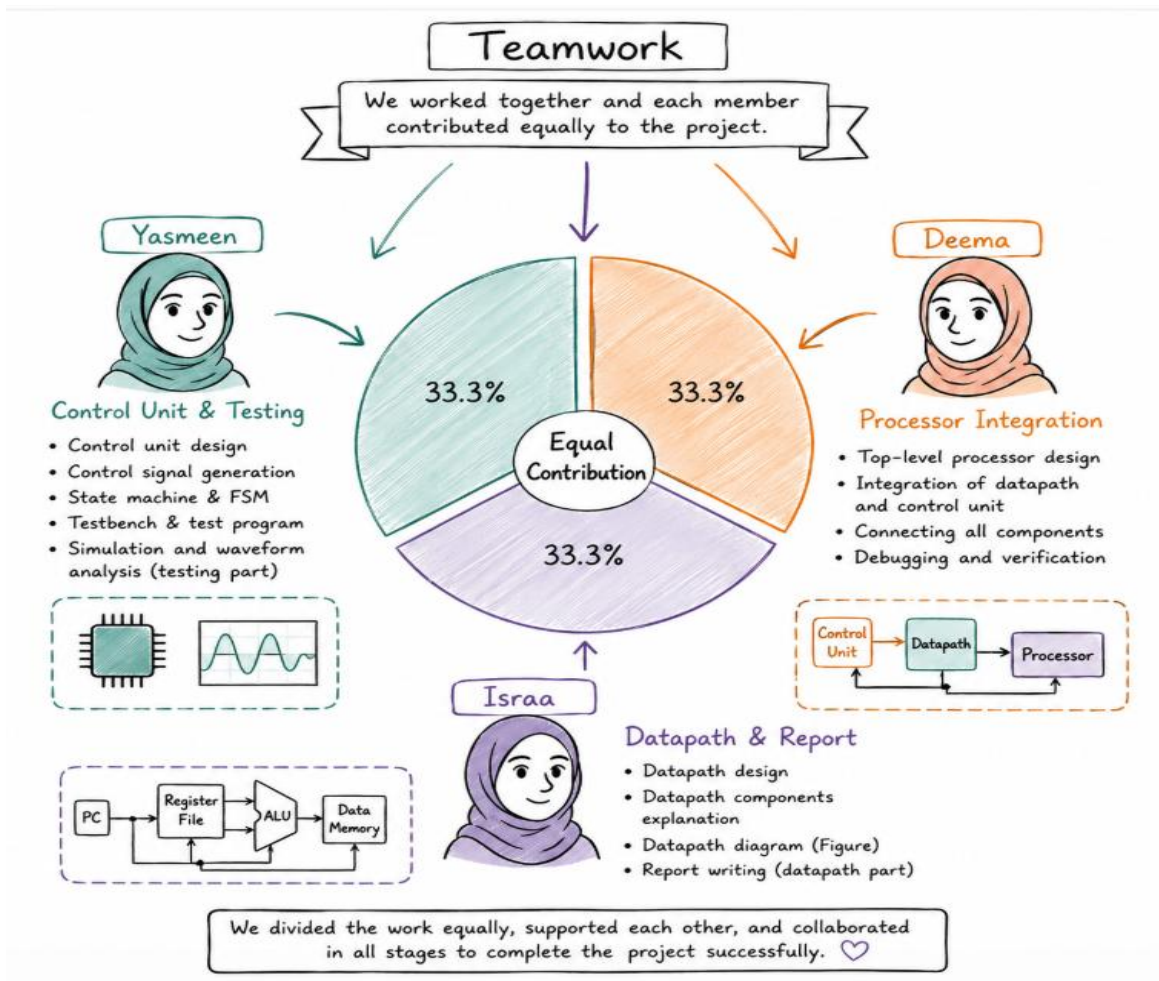


Figure 30 : Teamwork Contribution Distribution

Figure shows the teamwork distribution among the three team members. The work was divided equally, with each member contributing approximately one third of the total project effort. Yasmeen focused on the control unit, control signal generation, FSM, testbench, and waveform analysis. Deema worked on the top-level processor integration, connecting the datapath and control unit, and debugging the complete design. Israa worked on the datapath design, datapath components explanation, datapath diagram, and the related report sections. This distribution helped the team organize the work clearly while maintaining equal contribution and cooperation throughout the project.

Conclusion

This project presented the design and verification of a 32-bit multicycle RISC processor using Verilog HDL. The processor supports R-type, I-type, and J-type instructions, including arithmetic, logical, memory, branch, jump, and link operations. The datapath was designed using modular components such as the PC, instruction register, register file, ALU, extender, memories, multiplexers, and intermediate registers. A Moore FSM control unit was used to generate the required control signals and sequence each instruction through the proper execution stages. Simulation results verified that the processor components work correctly together and that the required instruction set is executed as intended. Overall, the project strengthened the understanding of processor datapath design, control unit implementation, and RTL verification.